A PROJECT REPORT

on

CHURNIQ

Customer Churn Prediction & Business Intelligence System

Submitted in partial fulfilment of the requirements
for the award of the degree of

[DEGREE — e.g. BACHELOR OF TECHNOLOGY]

in

[COURSE / BRANCH]

Submitted by

[STUDENT NAME] **[ROLL NUMBER]**

Under the guidance of

[FACULTY MENTOR NAME]
[DESIGNATION], [DEPARTMENT NAME]

[DEPARTMENT NAME]
[COLLEGE NAME]
[UNIVERSITY NAME]

[ACADEMIC SESSION / YEAR]

CERTIFICATE

This is to certify that the project report entitled **"ChurnIQ — Customer Churn Prediction & Business Intelligence System"** is a bona fide record of the work carried out by **[STUDENT NAME]**, bearing Roll Number **[ROLL NUMBER]**, a student of **[COURSE / BRANCH]** at **[COLLEGE NAME]**, in partial fulfilment of the requirements for the award of the degree of **[DEGREE]** under **[UNIVERSITY NAME]**.

The work presented in this report was carried out by the candidate under my supervision during the academic session **[ACADEMIC SESSION / YEAR]**. To the best of my knowledge, the content of this report has not been submitted previously to any other institution or university for the award of any degree or diploma.

_____

[FACULTY MENTOR NAME]
[DESIGNATION]
[DEPARTMENT NAME]
Project Guide

_____

[HEAD OF DEPARTMENT NAME]
Head of Department
[DEPARTMENT NAME]
H.O.D.

_____

[EXTERNAL EXAMINER]
External Examiner

Date: [DATE OF SUBMISSION]
Place: [PLACE]

DECLARATION

I, **[STUDENT NAME]**, bearing Roll Number **[ROLL NUMBER]**, hereby declare that the project report entitled **"ChurnIQ — Customer Churn Prediction & Business Intelligence System"**, submitted to **[COLLEGE NAME]**, **[UNIVERSITY NAME]**, in partial fulfilment of the requirements for the award of the degree of **[DEGREE]** in **[COURSE / BRANCH]**, is an original record of the work carried out by me under the guidance of **[FACULTY MENTOR NAME]**.

I further declare that:

- The design, source code, data-processing modules, machine-learning workflow and dashboard implementation described in this report were developed by me.
- All external libraries, datasets and reference material used in the project have been duly acknowledged and cited in the References section.
- The dataset used for model development is the publicly available IBM Telco Customer Churn dataset, which has been used strictly for academic purposes.
- The results, metrics and outputs reported in this document were obtained by executing the project source code and have not been altered or fabricated.
- This report, in whole or in part, has not been submitted to any other institution or university for the award of any degree or diploma.

Date: [DATE OF SUBMISSION]
Place: [PLACE]

_____

[STUDENT NAME]
[ROLL NUMBER]
[COURSE / BRANCH]

ACKNOWLEDGEMENT

The successful completion of this project would not have been possible without the guidance, encouragement and support extended to me by several individuals, and I take this opportunity to express my sincere gratitude to all of them.

I would first like to thank my project guide, **[FACULTY MENTOR NAME]**, [DESIGNATION], [DEPARTMENT NAME], for the continuous guidance, constructive feedback and technical direction provided throughout the development of ChurnIQ. The suggestions received during review discussions helped me refine the machine-learning workflow, improve the structure of the source modules and present the analytical results in a meaningful manner.

I extend my sincere thanks to **[HEAD OF DEPARTMENT NAME]**, Head of the Department of [DEPARTMENT NAME], for providing the academic environment, laboratory facilities and encouragement required to carry out this work.

I am also grateful to the faculty members and technical staff of **[COLLEGE NAME]** for their assistance and for the knowledge imparted during the course of study, which formed the foundation of this project.

I acknowledge the developers and maintainers of the open-source libraries used in this project — Python, pandas, NumPy, scikit-learn, Plotly and Streamlit — whose tools made this implementation possible. I also acknowledge IBM for publishing the Telco Customer Churn dataset used for model development.

Finally, I would like to thank my family and friends for their patience, motivation and constant support during the course of this work.

[STUDENT NAME]
[ROLL NUMBER]
[COURSE / BRANCH]

ABSTRACT

Customer churn — the discontinuation of a service by an existing customer — is a recurring problem for subscription-based businesses, where retaining an existing customer is generally less expensive than acquiring a new one. Conventional reporting tools describe churn only after it has occurred and offer little guidance on which individual customers require attention. This project addresses that gap by developing **ChurnIQ**, an end-to-end customer churn prediction and business intelligence system that converts raw customer data into customer-level risk intelligence and prioritised retention actions.

ChurnIQ is implemented in Python and organised into eleven source modules under a `Src/` package, together with a Streamlit application (`app.py`) that serves as the user interface. The system implements a complete data-science workflow: dataset validation for CSV, XLS and XLSX uploads; data cleaning; exploratory data analysis; a scikit-learn preprocessing pipeline combining median imputation and standard scaling for numerical attributes with most-frequent imputation and one-hot encoding for categorical attributes; supervised model training; model evaluation; churn prediction; customer risk scoring and segmentation; and rule-based retention recommendations.

Six classification algorithms — Logistic Regression, Decision Tree, Random Forest, Extra Trees, Gradient Boosting and Histogram-based Gradient Boosting — were trained and compared on the IBM Telco Customer Churn dataset (7,043 customer records, 21 attributes, churn rate 26.54%). Using an 80:20 stratified train–test split with a fixed random state, the training module ranks models by F1 score and persists the selected pipeline. The saved model is **Random Forest**, which achieved an accuracy of 76.08%, recall of 74.33%, F1 score of 62.26% and ROC-AUC of 83.69% on the held-out test set. Gradient Boosting recorded the highest accuracy (80.62%) and ROC-AUC (84.37%) but substantially lower recall (52.14%), and the trade-off between these two selection criteria is analysed in the report.

The prediction layer applies a three-tier strategy so that datasets other than the reference dataset can also be analysed: the saved model is used when the uploaded data is compatible, an adaptive model is trained when a churn label is present but the schema differs, and a heuristic behavioural risk engine is used when no label is available. Predicted probabilities are converted into a 0–100 risk score, four risk levels and six strategic customer segments, which are then mapped to retention priorities and engagement channels. On the reference dataset the pipeline classified 963 customers as Critical and 1,349 as High risk, and estimated the associated revenue exposure.

Results are presented through an eleven-page Streamlit dashboard supporting dataset upload, data-quality inspection, churn analytics, prediction, risk analysis, segmentation, customer-level exploration, model comparison, recommendations and CSV report downloads. The project demonstrates how a predictive model can be embedded within an interactive analytical application so that statistical output is translated into information that supports retention decision-making.

Keywords: Customer churn prediction, machine learning, classification, scikit-learn, customer segmentation, risk scoring, business intelligence, Streamlit dashboard, exploratory data analysis, retention analytics.

TABLE OF CONTENTS

LIST OF FIGURES

LIST OF TABLES

CHAPTER 1
INTRODUCTION

1.1 Background

Subscription-based and service-oriented industries such as telecommunications, banking, insurance and software-as-a-service depend on a stable base of recurring customers. In these industries the revenue generated by a customer accumulates over the period for which the customer remains active, and the departure of a customer therefore represents the loss of both current and future revenue. The event in which a customer discontinues a service is commonly referred to as *churn*.

Because the cost of acquiring a new customer is generally higher than the cost of retaining an existing one, organisations have an economic interest in identifying customers who are likely to leave before they actually do. Historical customer records — demographic attributes, subscribed services, contract terms, billing information and account tenure — contain measurable patterns that distinguish customers who left from those who stayed. Supervised machine learning provides a systematic way of learning these patterns from labelled historical data and applying them to the current customer base.

However, a predictive model on its own is of limited practical use. The output of a classifier is a probability, and a business user requires this probability to be expressed in terms of which customers need attention, how urgent that attention is, and what action might be appropriate. The present project was undertaken to build a system that spans this entire distance — from raw customer data to a prioritised, interpretable retention view.

1.2 Problem Statement

Organisations frequently possess sufficient customer data to anticipate churn, yet the data is examined only in aggregate through static reports. Such reports state how many customers were lost during a period but do not indicate which individual customers are currently at risk. Where predictive models are developed, they often remain in notebooks that are inaccessible to non-technical users, and their output is not linked to any prioritisation or recommendation mechanism.

The problem addressed by this project is therefore stated as follows: **to design and implement an integrated system that accepts customer data, validates and prepares it automatically, predicts the likelihood of churn for each customer using trained classification models, converts these predictions into risk levels and customer segments, derives retention recommendations from the resulting intelligence, and presents the complete analysis through an interactive dashboard that requires no programming knowledge to operate.**

1.3 Motivation

The motivation for this project arises from three considerations. The first is practical relevance: churn prediction is a well-defined problem with direct commercial value, which makes it a suitable subject for applying data-science techniques to a realistic scenario rather than an artificial exercise.

The second is the observation that the analytical value of a model is realised only when its output reaches the people who act on it. Building a dashboard around the model, rather than presenting the model alone, converts a technical artefact into a usable tool and demonstrates the complete lifecycle of a data-science solution.

The third is the opportunity to gain hands-on experience with an end-to-end workflow — data validation, cleaning, exploratory analysis, feature engineering, model training and comparison, model persistence, inference, and application development — within a single coherent project, and to handle the engineering problems that arise when these stages must operate together reliably.

1.4 Objectives of the Project

The objectives defined for the project are listed in Table 1.1. Each objective corresponds to a component that was implemented in the repository.

Table 1.1: Objectives of the ChurnIQ project

No.	Objective	Implementing Module
1	Accept customer datasets in CSV, XLS and XLSX formats and validate their suitability for churn analysis	dataset_validator.py, app.py
2	Clean the dataset by standardising column names, removing duplicates and empty records, correcting data types and handling missing values	clean_data.py
3	Perform exploratory data analysis to identify patterns associated with churn	eda.py, dashboard_analytics.py
4	Prepare predictive features through a reusable preprocessing pipeline that prevents data leakage	feature_engineering.py
5	Train multiple classification algorithms on historical customer data	model_training.py
6	Evaluate and compare models using accuracy, precision, recall, F1 score and ROC-AUC	model_training.py
7	Select the best-performing model and persist it together with its preprocessing pipeline and metadata	model_training.py
8	Generate customer-level churn predictions and probabilities from the saved model	prediction_pipeline_backup.py
9	Convert predictions into a risk score, risk level and customer value estimate	segmentation.py
10	Group customers into strategic business segments	segmentation.py
11	Derive retention recommendations, priorities and engagement channels	recommendations.py
12	Present all results through an interactive dashboard with downloadable reports	app.py, style.css

1.5 Scope of the Project

The scope of the project is defined by what the repository actually implements.

1.5.1 Included in Scope

- Dataset upload and validation for CSV, XLS and XLSX files, with automatic detection of common column-name variations.
- Automated data cleaning covering duplicate removal, whitespace standardisation, numeric type conversion and missing-value treatment.
- Exploratory data analysis covering churn distribution, contract type, payment method, tenure, charges, services and demographic attributes.
- A scikit-learn preprocessing pipeline with separate numerical and categorical branches, fitted only on training data.
- Training and comparison of six supervised classification algorithms with stratified cross-validation.

- Persistence of the selected model pipeline, the preprocessing object and the full comparison table.
- A three-tier prediction strategy that also accommodates datasets whose schema differs from the reference dataset.
- Customer risk scoring, four-level risk classification, customer value estimation and six strategic segments.
- Rule-based retention recommendations with priorities, urgency scores and engagement channels.
- An eleven-page Streamlit dashboard with interactive Plotly charts and CSV report downloads.

1.5.2 Excluded from Scope

The following are not implemented in the current version and are discussed in Chapter 14 as future scope:

- User authentication, authorisation and role management.
- Automated or scheduled model retraining and drift monitoring.
- Model explainability techniques such as SHAP or LIME.
- Streaming or real-time prediction on live transactional data.
- Cloud deployment and horizontal scaling.
- Automated dispatch of retention communications to customers.

A MySQL persistence module (`Src/database.py`) is present in the repository and the dashboard displays its connection status, but the analytical workflow operates entirely on files and does not require a database; the application continues to function when MySQL is unavailable.

1.6 Significance of the Project

The significance of ChurnIQ lies in its integration rather than in any single component. Data validation, cleaning, analysis, modelling, inference, segmentation, recommendation and visualisation are frequently treated as separate exercises; in this project they are implemented as connected modules that operate on a shared data structure and are exposed through one interface.

From an academic perspective, the project demonstrates the complete supervised learning lifecycle, including the aspects that are often omitted from coursework — preventing data leakage by fitting the preprocessor only on training data, selecting an evaluation metric appropriate to an imbalanced target, persisting a model together with the metadata required to reuse it, and handling inputs that do not conform to the expected schema.

From a practical perspective, the system produces output that can be acted upon: a ranked list of customers with risk levels, segments, estimated revenue exposure and suggested engagement channels, all exportable as CSV files for use in downstream business processes.

1.7 Organisation of the Report

The remainder of this report is organised as follows. Chapter 2 describes the nature and schedule of the work. Chapter 3 explains the technologies used. Chapter 4 analyses the existing approach and specifies the requirements of the proposed system. Chapter 5 presents the system design, architecture and module structure. Chapter 6 describes the dataset and the preprocessing performed. Chapter 7 presents the exploratory data analysis. Chapter 8 covers the machine learning models, their evaluation and the prediction pipeline. Chapter 9 explains customer risk scoring and segmentation. Chapter 10 describes the Streamlit dashboard. Chapter 11 discusses the business insights and recommendation logic. Chapter 12 presents the testing performed. Chapter 13 reports and discusses the results. Chapter 14 states the limitations and future scope, and Chapter 15 concludes the report. References and four appendices follow.

CHAPTER 2
INTERNSHIP / PROJECT OVERVIEW

Note on this chapter. The organisational and scheduling details of an internship or project engagement are administrative records held by the student and the institution. They cannot be derived from the source repository, and no such information has been assumed. Placeholders in square brackets are provided below and must be completed before submission. The technical content of the chapter reflects the work actually present in the repository.

2.1 Nature of the Work

The work reported in this document consisted of the design and implementation of a customer churn prediction and business intelligence system. It was carried out as an individual software and data-science development activity, covering the complete lifecycle from data acquisition through to the delivery of a working interactive application.

The activity was primarily one of applied development. It required the study of the problem domain, examination of the reference dataset, implementation of data-processing and machine-learning modules in Python, empirical comparison of several classification algorithms, and construction of a web-based dashboard to present the results. The deliverable is the ChurnIQ repository, which contains approximately 22,000 lines of Python and CSS across eleven source modules, an application entry point, a stylesheet, dataset files and saved model artefacts.

2.2 Organisation Details

Table 2.1: *Organisation details (to be completed by the candidate)*

Field	Value
Organisation / Institute name	[ORGANIZATION NAME]
Address	[ORGANIZATION ADDRESS]
Department / Division	[DEPARTMENT / DIVISION]
Nature of organisation	[NATURE OF ORGANIZATION]
Industry mentor	[INDUSTRY MENTOR NAME]
Industry mentor designation	[INDUSTRY MENTOR DESIGNATION]
Faculty mentor	[FACULTY MENTOR NAME]
Faculty mentor designation	[DESIGNATION], [DEPARTMENT NAME]
Mode of work	[ONSITE / REMOTE / HYBRID]

2.3 Duration and Schedule

Table 2.2: *Duration and schedule (to be completed by the candidate)*

Field	Value
Start date	[INTERNSHIP START DATE]
End date	[INTERNSHIP END DATE]
Total duration	[TOTAL DURATION IN WEEKS]
Working days per week	[WORKING DAYS PER WEEK]
Daily working hours	[DAILY WORKING HOURS]
Reporting frequency	[REVIEW / REPORTING FREQUENCY]

2.4 Roles and Responsibilities

The responsibilities discharged during the development of the project were as follows:

- Studying the customer churn problem and identifying the analytical questions the system should answer.
- Acquiring and examining the IBM Telco Customer Churn dataset and assessing its structure and quality.
- Designing a modular project structure separating data processing, machine learning, business intelligence and presentation concerns.
- Implementing dataset validation and cleaning routines capable of handling multiple file formats and column-naming conventions.
- Implementing the exploratory data analysis module and the dashboard analytics module.
- Building the preprocessing pipeline and the model training and comparison workflow.
- Evaluating six classification algorithms and persisting the selected model with its metadata.
- Implementing the prediction pipeline, including fallback strategies for datasets with a different schema.
- Implementing customer risk scoring, segmentation and the recommendation engine.
- Developing the eleven-page Streamlit dashboard and the accompanying stylesheet.
- Testing the system against the reference dataset and various input conditions, and documenting the work in this report.

2.5 Phase-wise Execution Plan

The project was executed in ten phases. The phase numbering shown below is not an assumption of the author of this report: it is recorded in the module docstrings of the repository itself — for example, `eda.py` is annotated “Phase 3: EDA & Analytics”, `feature_engineering.py` as “Phase 4”, `model_training.py` as “Phase 5”, and `app.py` as “Phase 10 — Streamlit Dashboard”.

Table 2.3: Phase-wise execution plan

Phase	Activity	Primary Deliverable	Duration
1	Problem study, dataset acquisition and project setup	Repository structure, Data/Telco-Customer-Churn.csv	[WEEK RANGE]
2	Dataset validation and cleaning	dataset_validator.py, clean_data.py, telco_churn_clean.csv	[WEEK RANGE]
3	Exploratory data analysis	eda.py	[WEEK RANGE]
4	Feature engineering and preprocessing pipeline	feature_engineering.py, telco_churn_processed.csv	[WEEK RANGE]
5	Model training and comparison	model_training.py, model_results.csv	[WEEK RANGE]
6	Model evaluation and persistence	best_churn_model.pkl, preprocessing.pkl	[WEEK RANGE]
7	Prediction pipeline development	prediction_pipeline.py, prediction_pipeline_backup.py	[WEEK RANGE]
8	Customer intelligence and segmentation	segmentation.py, customer_segments.csv	[WEEK RANGE]
9	Recommendation engine and analytics	recommendations.py, dashboard_analytics.py	[WEEK RANGE]
10	Dashboard development, styling and testing	app.py, style.css	[WEEK RANGE]

2.6 Learning Outcomes

The following learning outcomes were achieved through the execution of the project:

- **Data engineering.** Reading heterogeneous file formats, detecting column semantics from name variations, converting mixed-type columns and treating missing values in a controlled manner.
- **Exploratory analysis.** Producing grouped churn-rate statistics, tenure banding, correlation analysis and outlier summaries, and interpreting them in business terms.
- **Machine learning practice.** Constructing a `ColumnTransformer`-based preprocessing pipeline, avoiding data leakage by fitting only on training data, applying class weighting to an imbalanced target, and using stratified cross-validation.
- **Model evaluation.** Understanding why accuracy alone is misleading on an imbalanced dataset and why recall and F1 score are more informative for churn detection.
- **Software engineering.** Structuring a project into cohesive modules, designing defensive interfaces that fail gracefully, and persisting models with the metadata needed for reuse.
- **Application development.** Building a multi-page Streamlit application, managing session state across pages, generating interactive Plotly charts and implementing file download functionality.

- **Technical documentation.** Recording design decisions, test cases and results in a structured report.

CHAPTER 3
TECHNOLOGIES AND TOOLS

3.1 Overview of the Technology Stack

The technologies described in this chapter were confirmed by examining the import statements of every source file in the repository and the contents of `requirements.txt`. Only components that are actually used are described. Table 3.1 summarises the stack.

Table 3.1: *Technology stack used in the project*

Layer	Technology	Role in ChurnIQ
Language	Python 3	Implementation language for all source modules
Data handling	pandas	DataFrame operations, grouping, aggregation, CSV and Excel input/output
Numerical computing	NumPy	Array operations, vectorised conditions (np.select), numerical utilities
Machine learning	scikit-learn	Preprocessing, pipelines, six classification algorithms, evaluation metrics, cross-validation
Model persistence	joblib	Serialising and loading the trained pipeline and preprocessing objects
Visualisation	Plotly Express / Graph Objects	Interactive charts rendered inside the dashboard
Application framework	Streamlit	Multi-page web application, widgets, session state, file upload and download
Spreadsheet support	openpyxl, xlrd	Reading .xlsx and .xls uploads
Optional database	mysql-connector-python	Optional MySQL persistence layer (Src/database.py)
Styling	CSS	Custom dashboard appearance via style.css
Version control	Git and GitHub	Source-code management and hosting

3.2 Programming Language

Python was selected as the implementation language because it provides a mature ecosystem for data analysis and machine learning, and because the same language can be used for data processing, model development and web application development. This avoids the need to transfer models between environments. Every source file in the repository is written in Python, with the exception of the stylesheet.

3.3 Data Processing Libraries

3.3.1 pandas

pandas is the principal data-handling library in the project and is imported by every source module. It is used to read CSV files through `pd.read_csv()` and Excel files through `pd.read_excel()`; to inspect structure via `shape`, `dtypes` and `isna()`; to convert mixed-type columns with `pd.to_numeric(errors='coerce')`; to compute grouped churn statistics with `groupby()` and `crosstab()`; to create tenure and charge bands with `pd.cut()`; and to export result tables with `to_csv()`.

3.3.2 NumPy

NumPy supports the numerical operations underlying the analytics and segmentation modules. It is used for array conversion and clipping in the prediction pipeline, for the vectorised conditional assignment of retention priorities and engagement channels through `np.select()`, for defining the boundaries of risk-level bins using `np.inf`, and for selecting numeric columns via `select_dtypes(include=np.number)`.

3.4 Machine Learning Library

scikit-learn provides the entire machine-learning capability of the project. The specific components imported by the repository are:

- `ColumnTransformer` — applies different preprocessing to numerical and categorical columns within a single object.
- `Pipeline` — chains the preprocessor and the estimator so that identical transformations are applied during training and inference.
- `SimpleImputer` — median imputation for numerical features and most-frequent imputation for categorical features.
- `StandardScaler` — standardises numerical features to zero mean and unit variance.
- `OneHotEncoder` — encodes categorical features, configured with `handle_unknown='ignore'` so that unseen categories at prediction time do not raise an error.
- `train_test_split` — creates the stratified 80:20 training and testing partitions.
- `StratifiedKFold` and `cross_val_score` — stratified cross-validation using F1 as the scoring metric.
- `VarianceThreshold` — imported in the feature engineering module for low-variance feature filtering.
- Estimators: `LogisticRegression`, `DecisionTreeClassifier`, `RandomForestClassifier`, `ExtraTreesClassifier`, `GradientBoostingClassifier` and `HistGradientBoostingClassifier`.
- Metrics: `accuracy_score`, `precision_score`, `recall_score`, `f1_score`, `roc_auc_score` and `confusion_matrix`.

3.5 Visualisation Library

Plotly is the visualisation library used by the dashboard. `plotly.express` and `plotly.graph_objects` are imported in `app.py`, and charts are rendered with `st.plotly_chart()`. The chart types used in the application are pie charts (churn composition, segment composition, recommended channels), bar charts (contract analysis, risk distribution, segment sizes, grouped model comparison) and histograms (churn probability distribution). Plotly was chosen over static plotting libraries because its output supports hovering, zooming and legend filtering directly in the browser.

Clarification. `matplotlib` and `seaborn` are listed in `requirements.txt`, but a search of the repository confirms that neither library is imported by any application source file. They are therefore declared dependencies rather than active components of the running system, and no claim is made in this report that the dashboard renders matplotlib or seaborn charts. (The static figures printed in this report were produced by a separate report-generation script that reads the same repository data.)

3.6 Application Framework

Streamlit provides the user interface. It was chosen because it allows a data-oriented web application to be written entirely in Python without separate front-end code. The Streamlit features used in `app.py` include:

- `st.set_page_config()` — sets the page title, icon, wide layout and initial sidebar state.
- `st.sidebar.radio()` — renders the eleven-item navigation menu.
- `st.file_uploader()` — accepts dataset uploads restricted to `csv`, `xls` and `xlsx`.
- `st.session_state` — retains the active dataset and analysis results across page changes and reruns.
- `st.metric()`, `st.dataframe()`, `st.columns()` — KPI tiles, interactive tables and layout control.
- `st.download_button()` — exports result tables as CSV files.
- `st.spinner()`, `st.success()`, `st.warning()`, `st.error()` — progress and status feedback.
- `st.markdown(..., unsafe_allow_html=True)` — injects the custom stylesheet and structured HTML blocks.

3.7 Model Persistence

joblib is used to serialise and restore trained objects. In `model_training.py` the selected pipeline is written to `Models/best_churn_model.pkl` as a dictionary containing the fitted pipeline, the model name, a metadata dictionary and a version number. The fitted preprocessor is stored separately in `Models/preprocessing.pkl`. The prediction modules load these artefacts with `joblib.load()`. joblib was preferred to the standard `pickle` module because it handles large NumPy arrays, which occur in tree ensembles, more efficiently.

3.8 File-Format Support

openpyxl and **xlrd** extend pandas to read spreadsheet files. `dataset_validator.load_dataset()` selects the engine according to the file extension — `openpyxl` for `.xlsx` and `xlrd` for `.xls` — and raises an explanatory `ImportError` if the required package is not installed. For CSV files the loader attempts four encodings in sequence — `utf-8`, `utf-8-sig`, `latin1` and `cp1252` — so that files exported from different systems can be read without manual conversion.

3.9 Optional Database Support

mysql-connector-python is used by `Src/database.py`, which provides functions to open a connection, test connectivity, create customer and system tables, insert customer records, retrieve customers, count records, clear the table, initialise the schema and save an entire DataFrame. Connection parameters are read from the environment variables `CHURNIQ_DB_HOST`, `CHURNIQ_DB_PORT`, `CHURNIQ_DB_USER`, `CHURNIQ_DB_PASSWORD` and `CHURNIQ_DB_NAME`, with defaults defined in the module.

The dashboard imports this module inside a `try/except` block and displays the connection status in the sidebar as “MySQL Connected”, “MySQL Offline” or “MySQL Module Unavailable”. The analytical workflow does not depend on the database: all prediction, segmentation and reporting operate on in-memory DataFrames and files.

Security observation. The default value of the database password is currently written as a literal string inside `Src/database.py`. Although the module reads the value from an environment variable when one is set, storing a default credential in version-controlled source code is not good practice. This is recorded as a limitation in Chapter 14.

3.10 Styling and User Interface

CSS is used to customise the dashboard's appearance. The file `style.css` contains approximately 3,000 lines defining around 250 rule blocks, and is injected at start-up by the `load_css()` function in `app.py`. The stylesheet defines the hero banner, section titles and captions, KPI and insight cards, status indicators and table styling. The sidebar can additionally be hidden or shown at runtime through a toggle button that injects a rule targeting `section[data-testid="stSidebar"]`.

3.11 Version Control

Git was used for version control and **GitHub** for hosting the repository. A `.gitignore` file excludes the virtual environment directory, Python bytecode caches, `.streamlit/secrets.toml` and `.env` from version control.

3.12 Declared Dependencies

Table 3.2 reproduces the contents of `requirements.txt` and states whether each package is imported by the application source code.

Table 3.2: *Declared dependencies in requirements.txt*

Package	Purpose	Imported by application code
pandas	Data manipulation and analysis	Yes — all modules
numpy	Numerical computation	Yes — most modules
scikit-learn	Machine learning	Yes — feature_engineering, model_training, prediction pipelines
matplotlib	Static plotting	No — declared but not imported
seaborn	Statistical plotting	No — declared but not imported
plotly	Interactive charts	Yes — app.py
streamlit	Web application framework	Yes — app.py
mysql-connector-python	MySQL connectivity	Yes — database.py (optional)
openpyxl	Reading .xlsx files	Yes — dataset_validator, app.py
joblib	Model serialisation	Yes — feature_engineering, model_training, prediction pipelines
xlrd	Reading .xls files	Yes — dataset_validator, app.py

CHAPTER 4
SYSTEM ANALYSIS

4.1 Existing System

In the absence of a predictive system, churn is typically monitored through periodic reporting. Customer records are extracted from operational systems into spreadsheets, and summary statistics such as the number of customers lost during a month, the overall churn percentage and departures by region or plan are computed manually or through a standard reporting tool. Analysts may cross-tabulate churn against attributes such as contract type or payment method to identify groups with elevated loss.

Where analytical modelling is attempted, it is usually performed outside the reporting process. A data analyst prepares a dataset in a notebook environment, trains a classifier and reports an accuracy figure. The notebook is rarely integrated with the reporting workflow, and its results are communicated as static tables or slides.

4.2 Problems with the Existing Approach

- **Retrospective rather than predictive.** Reports describe customers who have already left. By the time a customer appears in a churn report, the opportunity to retain them has passed.
- **Aggregate rather than individual.** Knowing that month-to-month customers churn more frequently does not indicate *which* month-to-month customers are at risk, so retention effort cannot be targeted.
- **Manual and error-prone preparation.** Cleaning, type correction and missing-value handling performed by hand in spreadsheets is time-consuming and difficult to reproduce consistently.
- **No prioritisation.** Even when a list of at-risk customers exists, there is no mechanism to rank customers by the combination of risk and value, so limited retention budgets cannot be allocated efficiently.
- **Models isolated from users.** A model that exists only in a notebook cannot be operated by the business users who need its output.
- **Misleading evaluation.** Because churn datasets are imbalanced, a model reported only in terms of accuracy may appear satisfactory while failing to detect a large proportion of actual churners.
- **No linkage to action.** A probability value does not, by itself, indicate what should be done about a customer.

4.3 Proposed System

ChurnIQ is proposed as an integrated system that addresses each of these problems within a single application. The proposed system:

- Accepts a customer dataset through a browser interface in CSV, XLS or XLSX format, or uses the bundled reference dataset.
- Validates the upload automatically, detecting the churn target and key customer attributes despite differences in column naming, and reporting data quality.
- Cleans and prepares the data programmatically so that the same procedure is applied every time.
- Presents exploratory analytics that explain the distribution of churn across customer attributes.
- Applies a trained classification pipeline to produce a churn probability for every individual customer.
- Converts probabilities into a 0–100 risk score, one of four risk levels, an estimate of customer value and one of six strategic segments.
- Ranks customers by retention urgency, combining risk with estimated revenue exposure.
- Generates a specific recommended action and engagement channel for each customer using documented rules.
- Reports the full model comparison so that the basis of the prediction is visible rather than hidden.
- Allows every result table to be exported as a CSV file.

4.4 Advantages of the Proposed System

Table 4.1: *Comparison of the existing approach with the proposed system*

Problem in Existing Approach	How ChurnIQ Addresses It
Analysis is retrospective	A trained classifier assigns a churn probability to each currently active customer before departure occurs
Results are aggregate only	Output is produced at customer level, with an identifier, probability, risk score, level and segment for every record
Data preparation is manual	Validation and cleaning are performed by dataset_validator.py and clean_data.py, giving a repeatable procedure
No prioritisation of effort	A retention urgency score combines risk score (70%) with relative revenue at risk (30%) and ranks customers accordingly
Model inaccessible to users	The model is embedded in a Streamlit dashboard operated entirely through the browser
Evaluation limited to accuracy	Five metrics plus cross-validated F1 are computed for all six models and displayed in the dashboard
No connection to action	recommendations.py maps risk, segment, contract, payment method and tenure to a specific recommended action and channel
Rigid input requirements	Column-alias detection and a three-tier prediction strategy allow datasets other than the reference dataset to be analysed

4.5 Functional Requirements

Table 4.1 lists the functional requirements of the system. Each requirement was verified against the repository implementation.

Table 4.2: *Functional requirements*

ID	Requirement	Implemented In	Status
FR-01	Upload a customer dataset in CSV, XLS or XLSX format	app.py (Dataset Center)	Implemented
FR-02	Load a bundled default dataset when no file is uploaded	app.py — load_default_dataset()	Implemented
FR-03	Validate the dataset and detect the churn target and key columns	dataset_validator.py	Implemented
FR-04	Report data quality: rows, columns, missing cells, duplicates	dataset_validator.py, dashboard_analytics.py	Implemented
FR-05	Preview the active dataset and display column information	app.py (Dataset Center)	Implemented
FR-06	Clean the dataset: duplicates, whitespace, types, missing values	clean_data.py	Implemented
FR-07	Produce exploratory statistics on churn by customer attribute	eda.py, dashboard_analytics.py	Implemented
FR-08	Build a preprocessing pipeline for numerical and categorical features	feature_engineering.py	Implemented
FR-09	Split data into stratified training and testing sets	feature_engineering.py	Implemented
FR-10	Train multiple classification algorithms	model_training.py	Implemented
FR-11	Evaluate models on accuracy, precision, recall, F1 and ROC-AUC	model_training.py	Implemented
FR-12	Perform stratified cross-validation	model_training.py	Implemented
FR-13	Select and persist the best model with its metadata	model_training.py	Implemented
FR-14	Generate customer-level churn predictions and probabilities	prediction_pipeline_backup.py	Implemented
FR-15	Fall back to an adaptive model or a behavioural risk engine for incompatible datasets	prediction_pipeline_backup.py	Implemented
FR-16	Compute a 0–100 customer risk score and assign a risk level	segmentation.py	Implemented
FR-17	Estimate customer value and revenue at risk	segmentation.py	Implemented
FR-18	Assign customers to strategic segments	segmentation.py	Implemented
FR-19	Generate retention recommendations, priorities and channels	recommendations.py	Implemented
FR-20	Display the model comparison table and chart	app.py (Model Performance)	Implemented
FR-21	Search and inspect individual customer records	app.py (Customer Explorer)	Implemented
FR-22	Export result tables as CSV files	app.py — download_csv()	Implemented

ID	Requirement	Implemented In	Status
FR-23	Display MySQL connection status without blocking the workflow	app.py, database.py	Implemented

4.6 Non-Functional Requirements

Table 4.3: *Non-functional requirements*

ID	Category	Requirement and Basis in the Implementation
NFR-01	Usability	The system is operated entirely through a browser with sidebar navigation and requires no programming knowledge; each page carries a descriptive caption.
NFR-02	Reliability	Optional modules are imported inside try/except blocks and availability flags are checked before use, so the failure of one component does not stop the application.
NFR-03	Robustness	Input functions validate for None, non-DataFrame types and empty frames before processing, and file reading attempts multiple encodings.
NFR-04	Maintainability	Functionality is separated into eleven cohesive modules with docstrings and consistent naming, so components can be modified independently.
NFR-05	Reusability	Column-alias detection allows the analytical modules to operate on datasets with different naming conventions.
NFR-06	Performance	Prediction on the reference dataset of 7,043 records completes within a few seconds on a standard laptop; progress is indicated by a spinner.
NFR-07	Consistency	Preprocessing and the estimator are stored in a single scikit-learn Pipeline, guaranteeing identical transformation at training and inference.
NFR-08	Portability	The system depends only on cross-platform Python packages and runs on Windows, macOS and Linux.
NFR-09	Transparency	The dashboard reports which prediction method was used and displays the complete model comparison table.
NFR-10	Configurability	Database parameters are read from environment variables, allowing deployment without code changes.

4.7 Hardware Requirements

Table 4.4: *Hardware requirements*

Component	Minimum	Recommended
Processor	Dual-core 2.0 GHz	Quad-core 2.5 GHz or higher
Memory (RAM)	4 GB	8 GB or higher
Storage	2 GB free space	5 GB free space
Display	1366 × 768	1920 × 1080 (wide dashboard layout)
Network	Required for installing packages	Broadband connection

The training workload is modest: the reference dataset contains 7,043 records and the largest estimators are ensembles of 300 trees. Training all six models completes on a standard laptop without specialised hardware, and no GPU is required.

4.8 Software Requirements

Table 4.5: *Software requirements*

Component	Specification
Operating system	Windows 10/11, macOS or a Linux distribution
Python	Python 3.8 or later
Package manager	pip
Python packages	As listed in requirements.txt (Table 3.2)
Web browser	Google Chrome, Mozilla Firefox, Microsoft Edge or Safari
Version control	Git
Code editor	Visual Studio Code or an equivalent editor
Database (optional)	MySQL Server, required only for the optional persistence module

4.9 Feasibility Study

4.9.1 Technical Feasibility

The system is technically feasible. All required capabilities are provided by mature open-source libraries, the dataset is of a size that can be processed comfortably in memory, and the feasibility of the approach is demonstrated by the fact that the system has been implemented and executes successfully.

4.9.2 Economic Feasibility

The project is economically feasible. Every component of the software stack is open source and available at no cost, the dataset is publicly available, and no specialised hardware or paid service is required. The only cost is development effort.

4.9.3 Operational Feasibility

The system is operationally feasible. It is used through a web browser with a sidebar menu, dataset upload is a single interaction, and results are presented as charts, tables and downloadable CSV files that can be used in existing business processes without further technical support.

CHAPTER 5
SYSTEM DESIGN

5.1 Overall Architecture

ChurnIQ follows a layered architecture in which each layer has a defined responsibility and communicates with adjacent layers through pandas DataFrames. This separation allows a module to be modified or replaced without affecting unrelated parts of the system. Figure 5.1 shows the architecture as it exists in the repository.

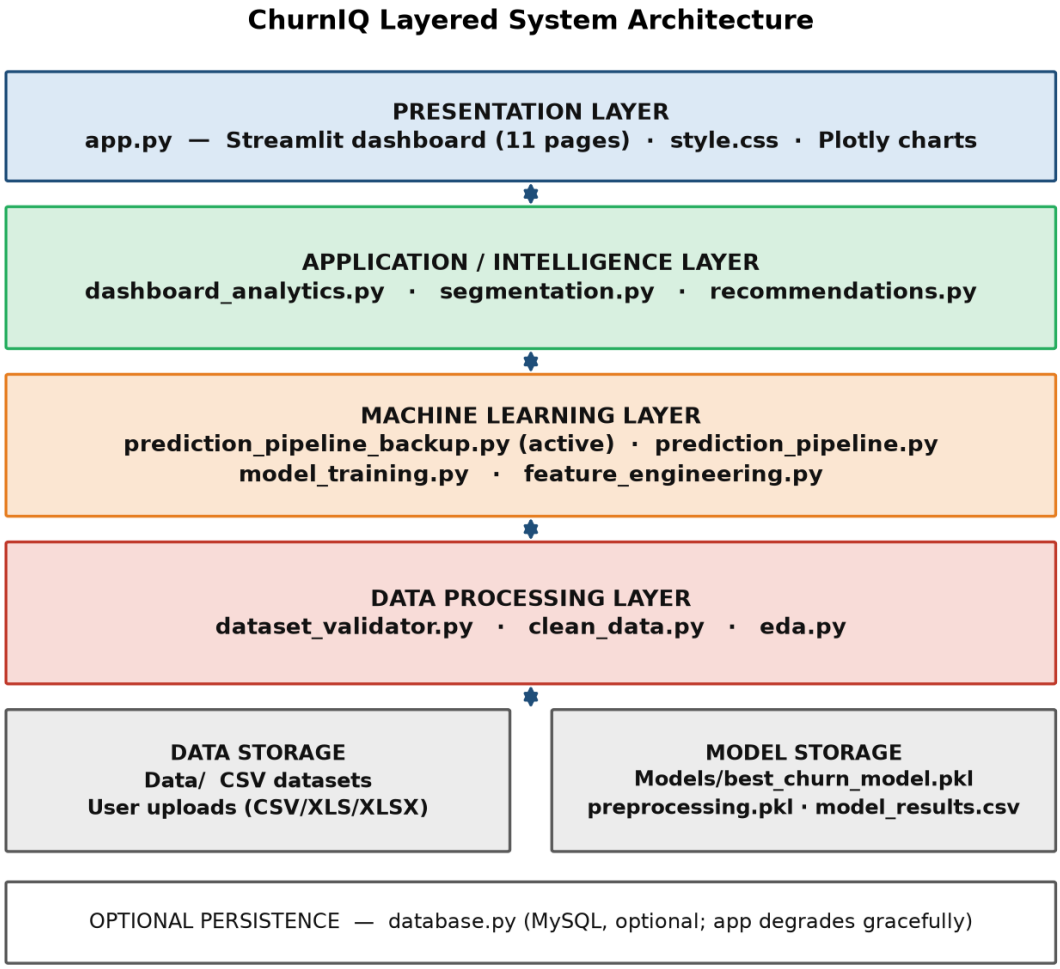

Figure 5.1: *ChurnIQ layered system architecture*

The **presentation layer** consists of `app.py`, which renders the eleven-page dashboard, and `style.css`, which controls its appearance. The **application layer** contains the modules that turn predictions into business meaning: `dashboard_analytics.py`, `segmentation.py` and `recommendations.py`. The **machine learning layer** contains the prediction pipelines, the training module and the feature-engineering module. The **data processing layer** contains validation, cleaning and exploratory analysis. Below these, datasets are held in `Data/` and model artefacts in `Models/`, with `database.py` providing optional MySQL persistence.

5.2 System Workflow

The workflow proceeds from raw data to business action in eleven stages, shown in Figure 5.2. Stages one to seven constitute the offline development workflow that produces the trained model; stages eight to eleven constitute the online workflow executed each time a user runs an analysis in the dashboard.

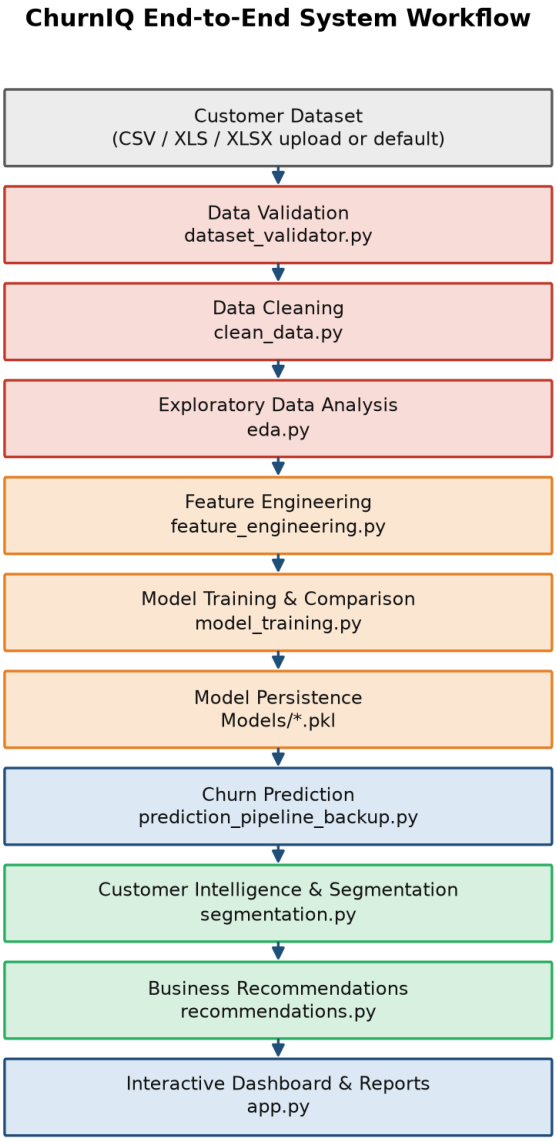

Figure 5.2: *ChurnIQ end-to-end system workflow*

5.3 Data Flow

Figure 5.3 presents the data flow of the system. The user supplies a dataset through the Dataset Center, or the application loads a default dataset from the `Data/` directory at start-up. The data is read and validated, cleaned and enriched with engineered features, then passed to the prediction process, which draws the trained pipeline from `Models/`. Predictions flow into risk scoring and segmentation, then into recommendation generation, and finally into the dashboard, which returns charts, tables and CSV exports to the user.

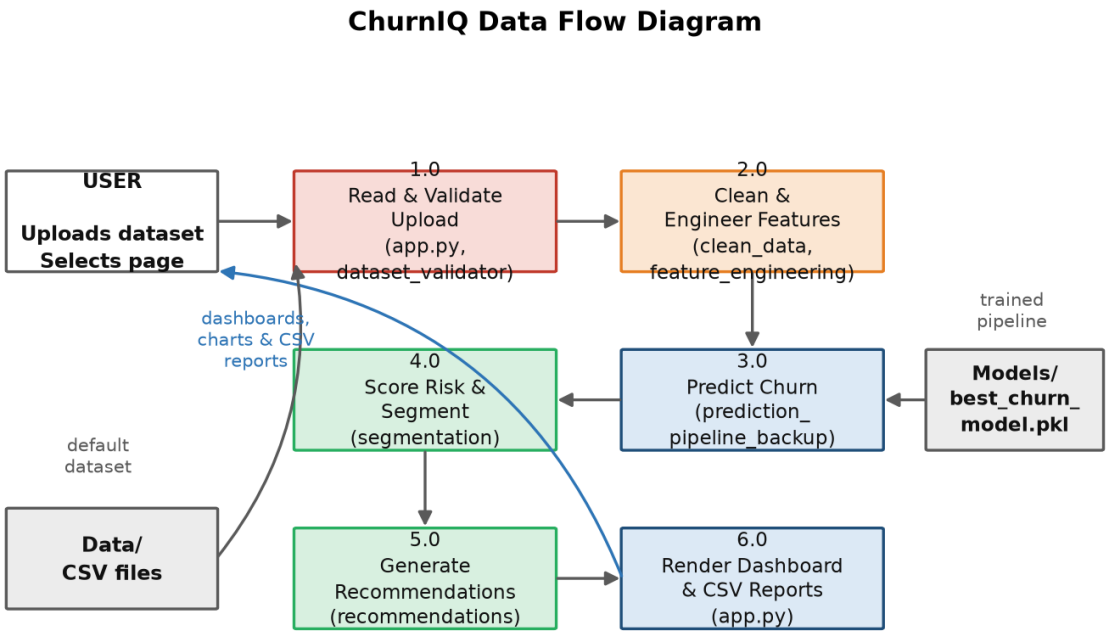

Figure 5.3: *ChurnIQ data flow diagram*

5.4 Data Processing Pipeline

Data processing is handled by three modules operating in sequence.

5.4.1 Validation

`dataset_validator.py` defines the set of supported extensions and a dictionary of column aliases covering seven logical fields: customer identifier, churn, tenure, monthly charges, total charges, contract and payment method. Column names are normalised by lower-casing and removing spaces, hyphens, underscores, slashes, dots and brackets, so that `MonthlyCharges`, `monthly_charges` and `Monthly Charge` all resolve to the same logical field. The validator then determines whether the dataset is “churn-ready”, which requires a detectable churn column and at least one useful customer feature.

5.4.2 Cleaning

`clean_data.py` exposes `clean_dataset()`, which applies seven operations in order: column-name cleaning, removal of entirely empty rows and columns, duplicate removal, text standardisation, numeric conversion, telecom-specific column correction and churn-target normalisation, followed by missing-value handling. The function returns the cleaned DataFrame together with a report recording the original and final row and column counts, the number of duplicates removed and the missing-value counts before and after cleaning.

5.4.3 Exploratory Analysis

`eda.py` provides read-only analysis functions and explicitly does not modify the input DataFrame. It produces the basic profile, churn analysis, missing-value analysis, numeric and

categorical summaries, churn rates grouped by any column, contract and payment analyses, tenure banding, a correlation matrix, an outlier summary and a set of generated textual insights.

5.5 Machine Learning Pipeline

The machine-learning design centres on a single scikit-learn `Pipeline` object that contains both the preprocessing step and the estimator. This is a deliberate design decision with two consequences. First, the transformations applied during inference are necessarily identical to those applied during training, eliminating a common source of error. Second, the preprocessor is fitted only when the pipeline is fitted on the training partition, so no information from the test partition influences imputation values or scaling parameters, thereby preventing data leakage. The construction of this pipeline is described in Section 6.7 and illustrated in Figure 6.1.

5.6 Dashboard Architecture

The dashboard is a single-file Streamlit application of approximately 5,200 lines. Its structure follows a fixed order: path configuration, page configuration, stylesheet loading, guarded module imports, session-state initialisation, data-loading helpers, utility functions, the sidebar toggle and navigation menu, the database status indicator, the hero banner, and then a chain of conditional blocks — one per page — followed by the footer.

Optional modules are imported defensively. Each import is wrapped in a `try/except` block that sets an availability flag such as `PREDICTION_AVAILABLE`, `SEGMENTATION_AVAILABLE`, `RECOMMENDATIONS_AVAILABLE`, `ANALYTICS_AVAILABLE` or `DATABASE_AVAILABLE`, together with the captured exception. Every page checks the relevant flag before calling into a module and displays an informative message if it is unavailable, so a missing dependency degrades a single feature rather than crashing the application.

5.7 Module Description

Table 5.1 describes every module in the repository, its size and its purpose. Figure 5.4 shows the dependency relationships between them.

Table 5.1: *Description of project modules*

Module	Lines	Purpose
app.py	5,232	Streamlit entry point; renders eleven dashboard pages, manages session state, orchestrates calls to the analytical modules and provides CSV downloads.
style.css	3,015	Custom stylesheet injected at start-up; defines the hero banner, section titles, KPI and insight cards, status indicators and table styling.
Src/dataset_validator.py	483	Loads CSV/XLS/XLSX files with encoding fallbacks, detects columns through aliases, performs quality checks and determines churn readiness.
Src/clean_data.py	429	Complete cleaning pipeline: column names, empty rows, duplicates, text standardisation, numeric conversion, telecom-specific corrections, target normalisation and missing values.
Src/eda.py	981	Read-only exploratory analysis: profiles, churn analysis, grouped churn rates, tenure bands, correlation, outliers and generated insights.
Src/feature_engineering.py	1,177	Target detection and binary conversion, feature-type preparation, removal of useless and identifier columns, ColumnTransformer construction, train/test split and preprocessor persistence.
Src/model_training.py	963	Defines six classifiers, builds preprocessing + model pipelines, trains and evaluates each, performs stratified cross-validation, selects the best model and saves artefacts.
Src/prediction_pipeline_backup.py	2,783	The prediction module imported by app.py. Implements cleaning, universal feature engineering, the three-tier prediction strategy, risk scoring and prediction summaries.
Src/prediction_pipeline.py	1,890	An alternative prediction implementation with model loading, compatibility checking, universal risk prediction and dataset intelligence. Present in the repository but not imported by app.py.
Src/segmentation.py	1,076	Computes the customer risk score, assigns risk levels, estimates customer value, assigns strategic segments, retention priority, priority rank and revenue at risk.
Src/recommendations.py	959	Generates a recommended action, retention recommendation, priority, urgency score and engagement channel per customer, plus business summaries and a priority action plan.
Src/dashboard_analytics.py	2,360	Supplies the dashboard with data quality metrics, model performance, executive insights, contract, tenure, charge, payment, service and demographic analyses, churn drivers, revenue at risk and KPIs.
Src/database.py	823	Optional MySQL layer: connection handling, table creation, record insertion and retrieval. Not required by the analytical workflow.

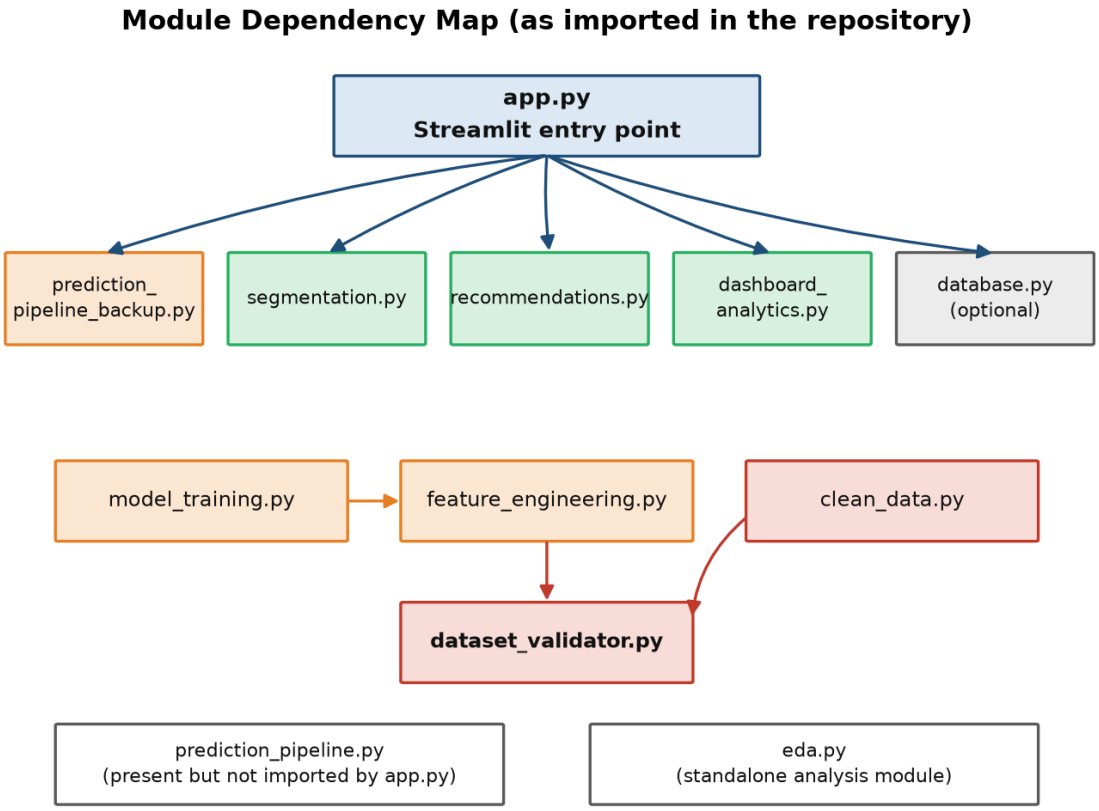

Figure 5.4: *Module dependency map showing imports as they exist in the repository*

Observation on the prediction modules. The repository contains two prediction implementations. The dashboard imports `predict_customer_churn` from `prediction_pipeline_backup.py`, and it is therefore this module — despite its name — that is executed at runtime. `prediction_pipeline.py` is present but is not imported by the application. Consolidating the two into a single module is identified as a maintenance improvement in Chapter 14.

5.8 Session-State Design

Streamlit re-executes the entire script on every user interaction, so any value that must survive between interactions has to be stored in `st.session_state`. ChurnIQ initialises the variables listed in Table 5.2 at start-up.

Table 5.2: *Session-state variables maintained by app.py*

Session Variable	Purpose
current_data	The active dataset, either uploaded by the user or loaded from the Data directory at start-up
data_source	The name of the file from which the active dataset was loaded, displayed in the Dataset Center
prediction_result	The DataFrame returned by the prediction pipeline, containing predictions, probabilities and risk fields
intelligence_result	The prediction result enriched by segmentation.py with risk score, value, segment and priority rank
recommendation_result	The intelligence result enriched by recommendations.py with actions, urgency score and channel
sidebar_hidden	Boolean controlling whether the navigation sidebar is displayed
last_upload_signature	Identifies the most recent upload so that the same file is not reprocessed on every rerun

A helper function, `get_result_data()`, returns the most complete result available, preferring the recommendation result, then the intelligence result, then the raw prediction result. This allows each page to request the richest available data without checking each variable individually. When a new dataset is uploaded, the three result variables are reset to `None` so that stale analysis from a previous dataset is never displayed.

CHAPTER 6
DATASET AND DATA PREPROCESSING

6.1 Dataset Description

The reference dataset used for model development is the **IBM Telco Customer Churn** dataset, stored in the repository as `Data/Telco-Customer-Churn.csv`. Each record represents one customer of a fictional telecommunications provider and describes the customer's demographic profile, the services subscribed to, the account and contract details, billing information and whether the customer left within the last month.

Table 6.1: *Dataset summary computed from the repository data file*

Property	Value
Dataset name	IBM Telco Customer Churn
File	Data/Telco-Customer-Churn.csv
Number of records	7,043
Number of attributes	21
Target variable	Churn (Yes / No)
Customers who churned	1,869 (26.54%)
Customers retained	5,174 (73.46%)
Duplicate records	0
Missing cells (as loaded)	0
Non-numeric values in TotalCharges	11 (blank strings)
Identifier column	customerID

The target is imbalanced: only 26.54% of customers churned. This imbalance is a central consideration in the modelling strategy, because a classifier that predicted “no churn” for every customer would still achieve an accuracy of 73.46% while identifying no churners at all. The treatment of this issue is described in Sections 8.4 and 8.8.

6.2 Dataset Features

Table 6.2 describes each attribute of the dataset.

Table 6.2: *Description of dataset features*

#	Feature	Type	Description
1	customerID	Categorical	Unique customer identifier; excluded from modelling
2	gender	Categorical	Female or Male
3	SeniorCitizen	Numeric (0/1)	Whether the customer is a senior citizen
4	Partner	Categorical	Whether the customer has a partner
5	Dependents	Categorical	Whether the customer has dependents
6	tenure	Numeric	Number of months the customer has stayed (0–72)
7	PhoneService	Categorical	Whether phone service is subscribed
8	MultipleLines	Categorical	Yes / No / No phone service
9	InternetService	Categorical	DSL / Fiber optic / No
10	OnlineSecurity	Categorical	Yes / No / No internet service
11	OnlineBackup	Categorical	Yes / No / No internet service
12	DeviceProtection	Categorical	Yes / No / No internet service
13	TechSupport	Categorical	Yes / No / No internet service
14	StreamingTV	Categorical	Yes / No / No internet service
15	StreamingMovies	Categorical	Yes / No / No internet service
16	Contract	Categorical	Month-to-month / One year / Two year
17	PaperlessBilling	Categorical	Whether paperless billing is used
18	PaymentMethod	Categorical	Electronic check / Mailed check / Bank transfer (automatic) / Credit card (automatic)
19	MonthlyCharges	Numeric	Amount charged monthly (18.25–118.75)
20	TotalCharges	Numeric (stored as text)	Total amount charged over the customer's lifetime
21	Churn	Categorical (target)	Yes if the customer left, No otherwise

6.3 Data Loading

Loading is implemented in `dataset_validator.load_dataset()`. The function verifies that the file exists and that its extension is one of `.csv`, `.xls` or `.xlsx`, raising a descriptive error otherwise. For CSV files it attempts four encodings in sequence so that files exported from different systems can be read.

```
encodings = ["utf-8", "utf-8-sig", "latin1", "cp1252"]

for encoding in encodings:
    try:
        df = pd.read_csv(path, encoding=encoding, low_memory=False)
        if len(df.columns) > 0:
            return df
    except (UnicodeDecodeError, pd.errors.ParserError) as error:
```

```
        last_error = error
```

The dashboard contains a parallel reader, `read_uploaded_file()`, that handles the in-memory object produced by `st.file_uploader()` and selects the appropriate engine by inspecting the file name. At start-up, `load_default_dataset()` attempts to load `telco_churn_clean.csv`, then `Telco-Customer-Churn.csv`, then `telco_churn_processed.csv`, so that the dashboard is populated even before the user uploads anything.

6.4 Data Validation

`validate_churn_dataset()` combines a basic quality check with column detection. The quality check computes the number of rows and columns, missing cells and the missing percentage, duplicate rows and entirely empty rows. Column detection resolves the seven logical fields through the alias dictionary described in Section 5.4.1.

A dataset is marked **churn-ready** when a churn column has been detected and at least one useful customer feature — tenure, monthly charges, total charges, contract or payment method — is present. Warnings are raised when no customer identifier is found, when no behavioural or billing feature is detected, when more than 30% of cells are missing, or when duplicate rows exist. The reference dataset passes validation with all seven logical fields detected and a computed quality score of 100%.

6.5 Missing Value Handling

Missing values are handled at two distinct points, which is an important design detail.

6.5.1 During Cleaning

`clean_data.handle_missing_values()` operates on the dataset as a whole. In the Telco dataset the `TotalCharges` column is stored as text and contains eleven blank entries, which correspond to customers with a tenure of zero months who have not yet been billed. When the column is converted to a numeric type these blanks become `NaN` values and are then imputed.

6.5.2 Inside the Model Pipeline

Imputation is also performed inside the preprocessing pipeline by `SimpleImputer`, using the median for numerical features and the most frequent value for categorical features. This second layer is essential rather than redundant: because the imputer is a step within the fitted pipeline, the imputation statistics are learned from the training partition only, and the same values are applied automatically to any new data presented at prediction time. This ensures that a future upload containing missing values is handled consistently without any separate preparation.

6.6 Data Cleaning

`clean_dataset()` applies the following operations in order:

- **Column-name cleaning** — strips leading and trailing whitespace and collapses repeated internal spaces, while preserving readable names.

- **Empty-data removal** — drops rows and columns in which every value is missing.
- **Duplicate removal** — removes duplicate rows and records how many were removed.
- **Text standardisation** — trims whitespace in object columns and converts the placeholder strings *“”*, *nan*, *None*, *NULL*, *null*, *N/A* and *n/a* into proper missing values.
- **Numeric conversion** — for text columns, removes currency symbols and thousands separators and converts the column to a numeric type when the overwhelming majority of its values parse as numbers.
- **Telecom-specific correction** — applies known corrections to common telecom columns such as TotalCharges.
- **Target normalisation** — converts the churn column into a consistent binary representation.
- **Missing-value handling** — imputes remaining gaps.

The function returns a report of the cleaning operation. Applying it to the raw dataset produces `Data/telco_churn_clean.csv`, in which the churn column is represented as 0 and 1 rather than “No” and “Yes”, while all 7,043 records and 21 columns are retained.

6.7 Data Transformation

Transformation is performed by the preprocessing pipeline constructed in `feature_engineering.build_preprocessor()`. The function partitions the feature matrix by data type and defines a separate sub-pipeline for each group, combining them in a `ColumnTransformer` with `remainder='drop'`. Figure 6.1 illustrates the resulting structure, and Table 6.3 lists the actual feature grouping recorded in the saved model metadata.

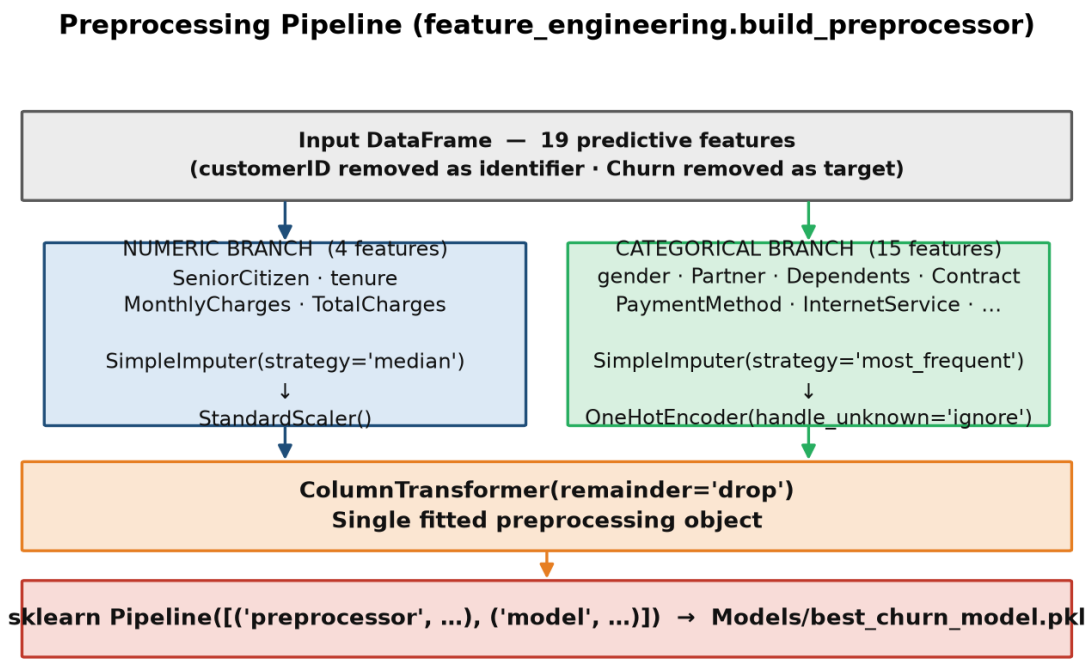

Figure 6.1: *Preprocessing pipeline built by feature_engineering.build_preprocessor()*

Table 6.3: *Feature grouping used by the preprocessor, as recorded in the model metadata*

Group	Count	Features	Transformations
Numerical	4	SeniorCitizen, tenure, MonthlyCharges, TotalCharges	SimpleImputer(strategy='median') → StandardScaler()
Categorical	15	gender, Partner, Dependents, PhoneService, MultipleLines, InternetService, OnlineSecurity, OnlineBackup, DeviceProtection, TechSupport, StreamingTV, StreamingMovies, Contract, PaperlessBilling, PaymentMethod	SimpleImputer(strategy='most_frequent') → OneHotEncoder(handle_unknown='ignore')
Excluded	2	customerID (identifier), Churn (target)	Removed before preprocessing

Standardisation is applied to numerical features so that attributes measured on different scales — tenure in months, charges in currency units — contribute comparably to distance- and gradient-based algorithms such as Logistic Regression. The encoder is configured with `handle_unknown='ignore'` so that a category not present during training does not cause a failure at prediction time; the corresponding indicator columns are simply set to zero.

6.8 Feature Engineering

Feature preparation in `feature_engineering.py` comprises target detection, target conversion, and the removal of columns that should not be used for learning.

6.8.1 Target Detection and Conversion

`find_churn_column()` locates the target using the alias dictionary, and `convert_target_to_binary()` converts it to 0 and 1. If the column is already numeric and contains only zeros and ones it is used directly; otherwise values are normalised to lower case and matched against a set of positive tokens such as *yes*, *true*, *churn* and *churned*. Values that cannot be interpreted become missing rather than being silently assigned to a class.

6.8.2 Removal of Unusable Columns

`identify_useless_columns()` marks for removal any column that is entirely empty, constant, or a raw index artefact such as `Unnamed: 0`, `index`, `row_index` or `RowNumber`. Constant columns are removed because a feature with a single value carries no discriminative information.

6.8.3 Identifier Removal

`identify_identifier_columns()` detects customer identifiers both through the alias dictionary and by testing normalised column names for the substrings `customerid`, `clientid`, `accountid`, `accountnumber` and `customerkey`. As the module docstring states, identifiers represent identity rather than behaviour; retaining them would allow the model to memorise individual records instead of learning generalisable patterns. For the reference dataset this step removes `customerID`, leaving 19 predictive features.

6.8.4 Universal Feature Engineering at Prediction Time

A second and separate form of feature engineering exists in `prediction_pipeline_backup.engineer_features()`, which is applied to uploaded

datasets. It creates missing-value indicator columns for partially populated fields and derives internal signal columns — `__cq_tenure`, `__cq_recency`, `__cq_usage`, `__cq_support`, `__cq_payment_risk` and `__cq_value` — by locating relevant columns through alias matching. These derived signals are used by the behavioural risk engine described in Section 8.10 and are not part of the trained model's feature set.

6.9 Encoding

Categorical attributes are encoded using one-hot encoding, in which each category becomes a binary indicator column. This scheme was chosen because the categorical attributes in the dataset are nominal: contract types and payment methods have no inherent order, so an ordinal encoding would impose a false ranking that a numerical model would attempt to exploit.

The effect of encoding can be observed by comparing the cleaned and processed dataset files in the repository. `telco_churn_clean.csv` retains the original 21 columns, whereas `telco_churn_processed.csv` contains 46 columns: four numerical attributes, the target, and 41 indicator columns such as `Contract_Month-to-month`, `Contract_One year`, `Contract_Two year`, `InternetService_Fiber optic` and `PaymentMethod_Electronic check`.

6.10 Train/Test Preparation

`prepare_training_data()` produces the training and testing partitions. The configuration constants defined at the top of `model_training.py` are `TEST_SIZE = 0.20` and `RANDOM_STATE = 42`. The split is stratified on the target so that the proportion of churners is preserved in both partitions, and the fixed random state makes the split reproducible.

Table 6.4: *Train/test partition parameters recorded in the saved model metadata*

Parameter	Value
Total records	7,043
Training records	5,634 (80%)
Testing records	1,409 (20%)
Class distribution (full dataset)	Retained (0): 5,174 · Churned (1): 1,869
Predictive features	19
Stratified split	Yes
Random state	42

These values were read directly from the metadata dictionary stored inside `Models/best_churn_model.pkl` and therefore reflect the partition actually used when the saved model was trained.

6.11 Derived Dataset Files

The repository contains five CSV files in the `Data/` directory, representing successive stages of the workflow.

Table 6.5: *Dataset files present in the Data directory*

File	Rows	Columns	Description
Telco-Customer-Churn.csv	7,043	21	The original IBM Telco dataset as obtained
telco_churn_clean.csv	7,043	21	Output of clean_data.py; churn encoded as 0/1
telco_churn_processed.csv	7,043	46	Fully encoded numerical matrix after one-hot encoding
customer_segments.csv	7,043	34	Prediction output enriched with risk scores, value fields and segments
customer_recommendations.csv	7,043	39	Segment output enriched with retention priorities, actions and value at risk

CHAPTER 7
EXPLORATORY DATA ANALYSIS

7.1 Purpose and Implementation

Exploratory data analysis was carried out to understand the structure of the dataset, to identify which customer attributes are associated with churn, and to inform the modelling strategy. Two modules in the repository implement this analysis. `Src/eda.py` provides general-purpose analysis functions, including `get_basic_profile()`, `get_churn_analysis()`, `get_churn_by_column()`, `get_tenure_analysis()`, `get_correlation_matrix()` and `get_outlier_summary()`. `Src/dashboard_analytics.py` provides the analyses consumed directly by the dashboard, including contract, tenure, monthly-charge, payment-method, internet-service, gender, senior-citizen and service analyses, together with `get_churn_drivers()` and `get_revenue_at_risk()`.

All statistics reported in this chapter were computed from `Data/Telco-Customer-Churn.csv` and are consistent with the values returned by these modules when executed on the same data.

7.2 Churn Distribution

Of the 7,043 customers in the dataset, 1,869 churned and 5,174 were retained, giving a churn rate of 26.54%. The classes are therefore imbalanced in an approximate ratio of 1:2.8.

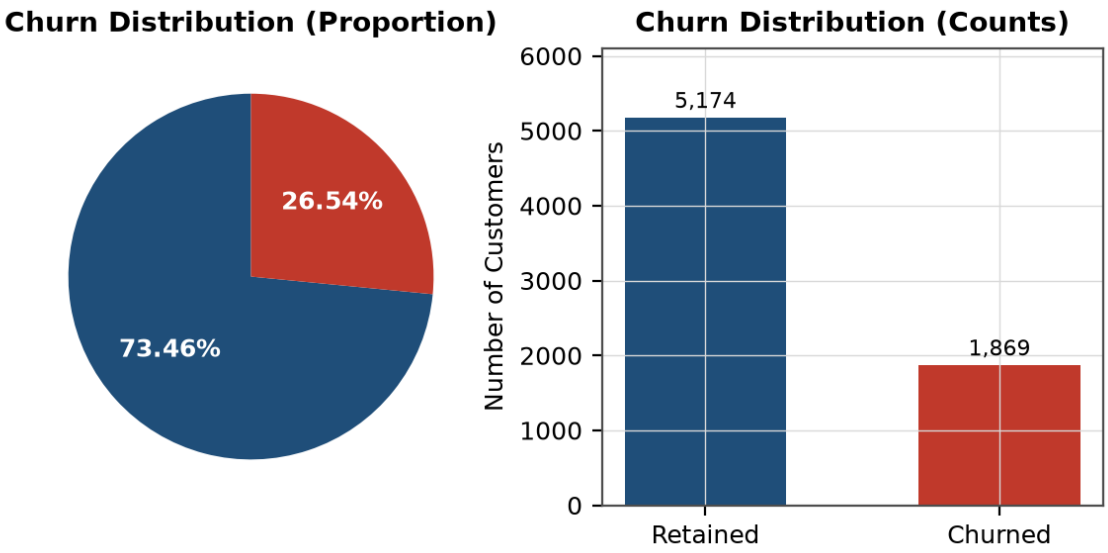

Figure 7.1: *Churn distribution in the reference dataset, shown as proportions and absolute counts*

This distribution has two consequences for the project. First, accuracy is an unreliable measure of quality, since predicting the majority class for every record would yield 73.46% accuracy while identifying no churners. Second, the models must be encouraged to attend to the minority class, which is achieved through class weighting and by ranking models on F1 score rather than accuracy.

7.3 Contract Type Analysis

Contract type shows the strongest association with churn of any attribute in the dataset.

Table 7.1: Churn rate by contract type

Contract Type	Customers	Churned	Churn Rate
Month-to-month	3,875	1,655	42.71%
One year	1,473	166	11.27%
Two year	1,695	48	2.83%

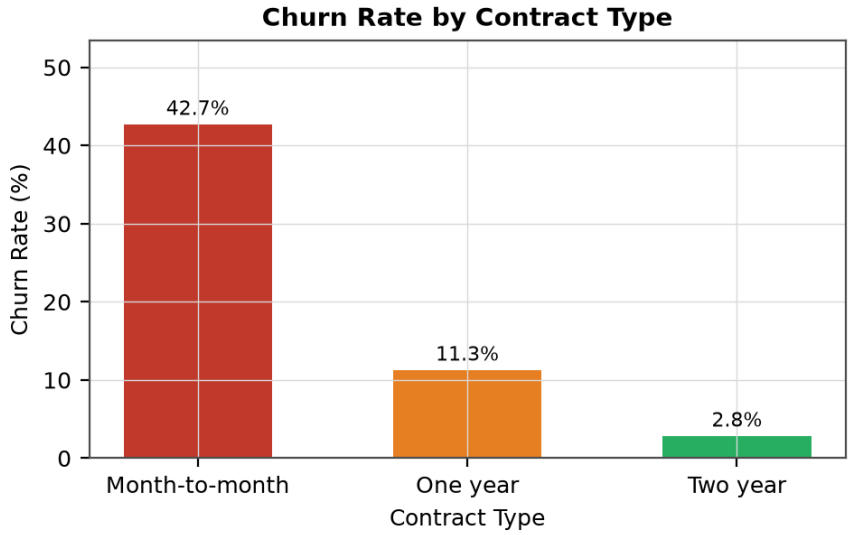

Figure 7.2: Churn rate by contract type

Customers on month-to-month contracts churn at 42.71%, approximately fifteen times the rate observed for two-year contracts (2.83%). The interpretation is straightforward: a month-to-month arrangement imposes no switching cost, so dissatisfaction can translate immediately into departure, whereas a long-term contract creates both a commitment and a natural review point. Month-to-month customers also constitute the largest group in the dataset (3,875 customers, 55% of the base), so this segment accounts for the majority of all churn.

7.4 Payment Method Analysis

Table 7.2: Churn rate by payment method

Payment Method	Customers	Churned	Churn Rate
Electronic check	2,365	1,071	45.29%
Mailed check	1,612	308	19.11%
Bank transfer (automatic)	1,544	258	16.71%
Credit card (automatic)	1,522	232	15.24%

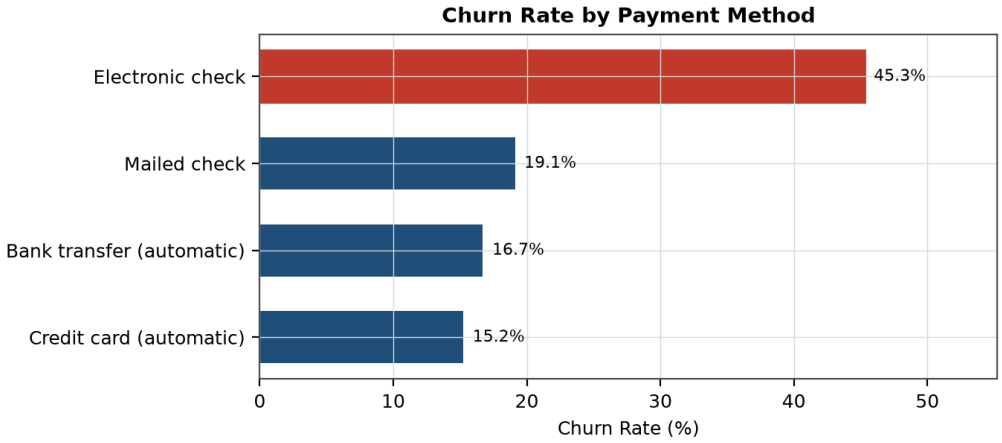

Figure 7.3: *Churn rate by payment method*

Customers paying by electronic check churn at 45.29%, nearly three times the rate of those using automatic credit-card payment (15.24%). The two automatic methods show the lowest churn. A plausible explanation is that manual payment requires a recurring active decision, each instance of which is an opportunity to reconsider the service, whereas automatic payment continues by default. This relationship is used directly in the recommendation logic described in Section 11.2, where high-risk customers paying by electronic check are advised to be offered an easier payment option.

7.5 Tenure Analysis

Tenure — the number of months a customer has remained with the provider — is strongly and inversely related to churn. The bands below follow those used by `dashboard_analytics.get_tenure_analysis()`.

Table 7.3: *Churn rate by tenure group*

Tenure Group	Customers	Churned	Churn Rate
0–6 months	1,481	784	52.94%
7–12 months	705	253	35.89%
13–24 months	1,024	294	28.71%
25–36 months	832	180	21.63%
37–60 months	1,594	265	16.62%
60+ months	1,407	93	6.61%

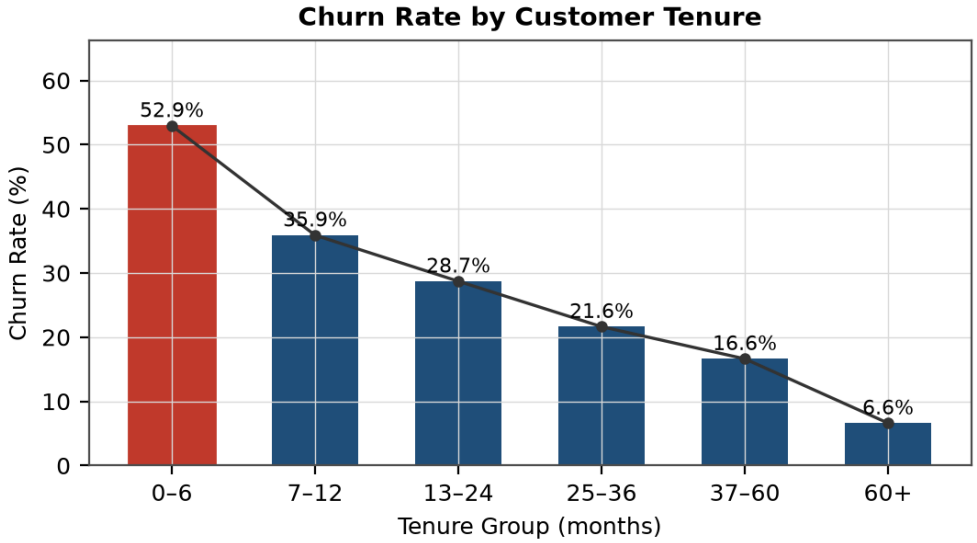

Figure 7.4: *Churn rate by customer tenure group*

Churn declines monotonically with tenure, from 52.94% in the first six months to 6.61% beyond sixty months. More than half of all customers in their first six months leave, and this single band accounts for 784 of the 1,869 churn events in the dataset — approximately 42% of all churn. The finding indicates that the early period of the customer relationship is the point of greatest vulnerability, and it is reflected in the recommendation engine, which proposes an onboarding and engagement campaign for high-risk customers whose tenure is six months or less.

7.6 Service-Related Analysis

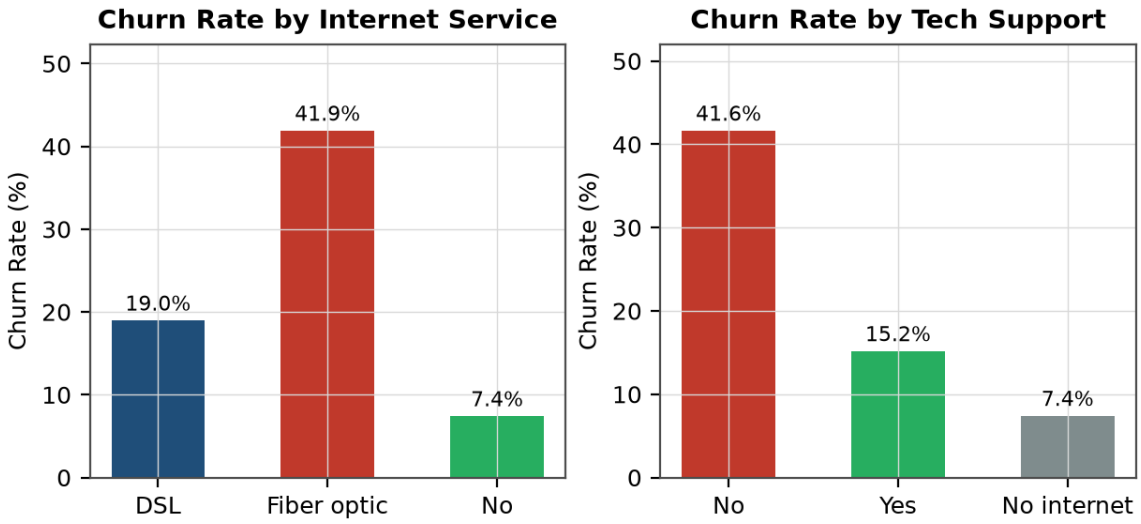

Figure 7.5: *Churn rate by internet service type and by availability of technical support*

Customers subscribing to fibre-optic internet churn at 41.89%, compared with 18.96% for DSL and 7.40% for customers with no internet service. Since fibre optic is the premium and more expensive product, the elevated churn among its subscribers suggests either an expectation gap between price and perceived service quality, or greater price sensitivity among customers paying more.

The absence of technical support is also associated with elevated churn: 41.64% of customers without technical support churned, compared with 15.17% of those with it. A similar pattern is observed for online security, where churn is 41.77% without the service and 14.61% with it. Two interpretations are possible and cannot be distinguished from this data alone: support services may genuinely reduce churn by resolving problems before they escalate, or customers who purchase additional services may simply be more committed to the provider. Both interpretations are consistent with treating the absence of these services as a risk indicator.

7.7 Demographic Analysis

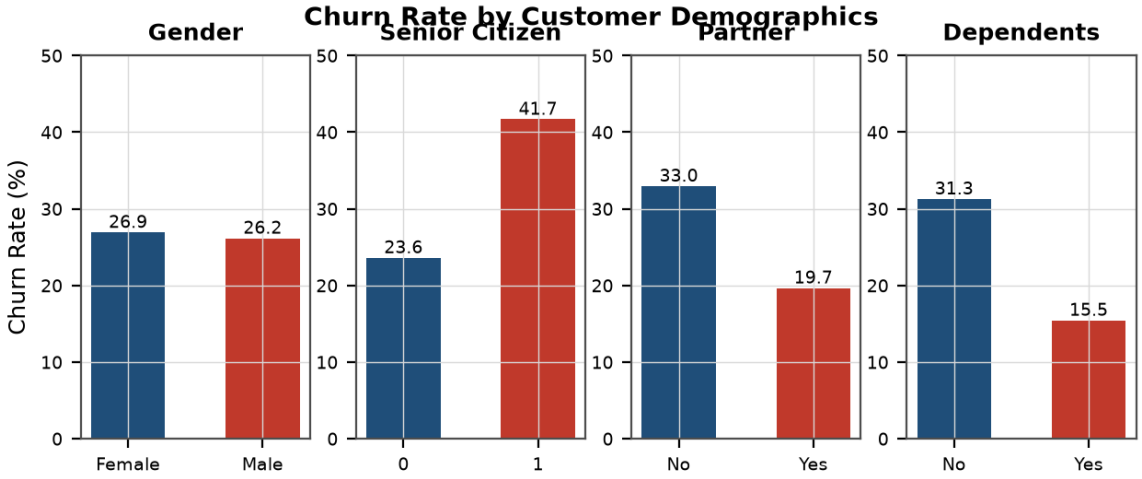

Figure 7.6: *Churn rate by gender, senior-citizen status, partner status and dependent status*

Gender shows almost no association with churn: 26.92% of female customers and 26.16% of male customers churned, a difference of less than one percentage point. This is a useful negative finding, as it indicates that gender carries little predictive information in this dataset.

The remaining demographic attributes are more informative. Senior citizens churned at 41.68% compared with 23.61% for non-seniors. Customers without a partner churned at 32.96% against 19.66% for those with one, and customers without dependents at 31.28% against 15.45% for those with dependents. The pattern suggests that customers with household attachments are more stable, possibly because a shared service is less readily changed than an individual one.

7.8 Numerical Feature Analysis

Table 7.4: Descriptive statistics of the numerical features

Statistic	tenure (months)	MonthlyCharges	TotalCharges
Count	7,043	7,043	7,032
Mean	32.37	64.76	2,283.30
Standard deviation	24.56	30.09	2,266.77
Minimum	0.00	18.25	18.80
25th percentile	9.00	35.50	401.45
Median	29.00	70.35	1,397.48
75th percentile	55.00	89.85	3,794.74
Maximum	72.00	118.75	8,684.80

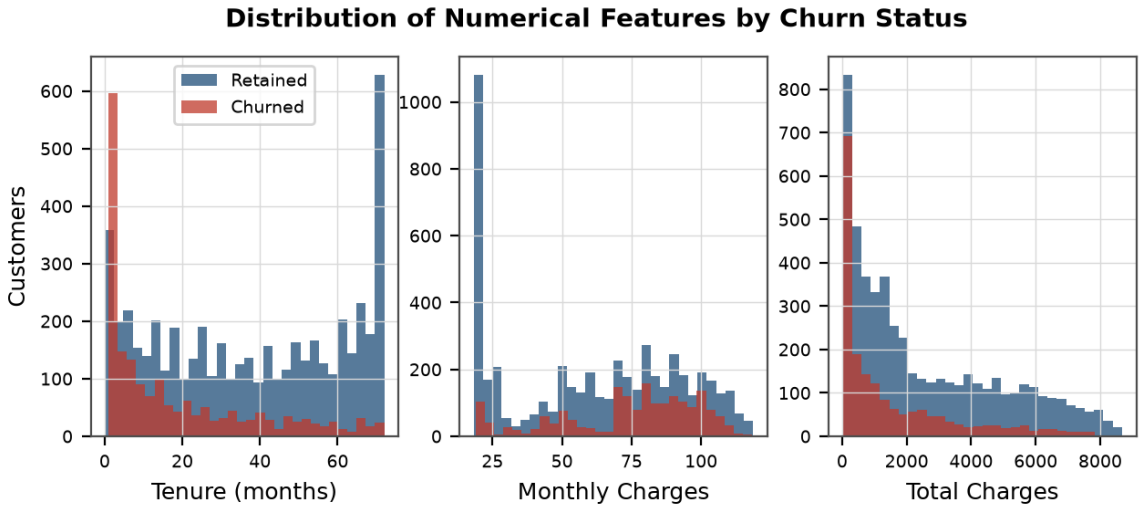

Figure 7.7: Distribution of tenure, monthly charges and total charges by churn status

The count for `TotalCharges` is 7,032 rather than 7,043 because eleven records contain a blank value in this column; these correspond to customers with zero tenure who have not yet been billed. This is the specific missing-value case handled by the cleaning and imputation steps described in Section 6.5.

Table 7.5: Mean values of numerical features by churn status

Churn Status	Mean tenure	Mean MonthlyCharges	Mean TotalCharges
Retained	37.57 months	61.27	2,555.34
Churned	17.98 months	74.44	1,531.80

Churned customers had a mean tenure of 17.98 months against 37.57 months for retained customers, and paid higher mean monthly charges (74.44 against 61.27). Their mean total charges are nevertheless lower (1,531.80 against 2,555.34), which is a direct consequence of their shorter tenure: they paid more per month but for fewer months. The charge analysis produced by `get_monthly_charge_analysis()` shows churn rising from 11.24% in the lowest charge band

to 37.51% in the high band, before easing slightly to 32.88% in the very-high band.

7.9 Correlation Analysis

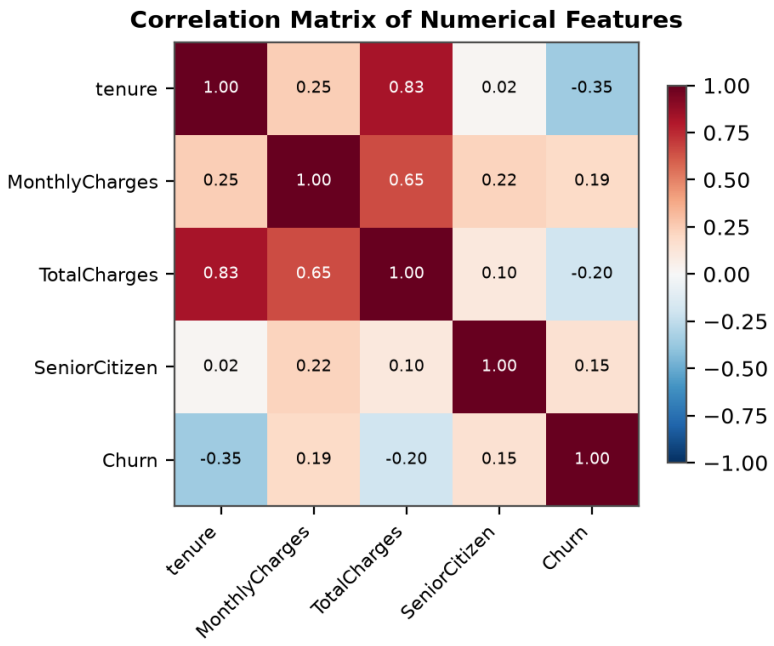

Figure 7.8: *Correlation matrix of the numerical features and the churn indicator*

Tenure shows the strongest negative correlation with churn among the numerical attributes, consistent with the tenure analysis in Section 7.5, while monthly charges show a positive correlation. Tenure and total charges are strongly positively correlated with one another, which is expected because total charges are approximately the product of monthly charges and tenure. This multicollinearity does not affect the tree-based ensembles, and the regularisation applied by Logistic Regression limits its impact there.

7.10 Churn Driver Ranking

`dashboard_analytics.get_churn_drivers()` ranks category-level churn contributions by an impact score. Applied to the reference dataset, the highest-ranked categories are two-year contracts, absence of internet service, electronic-check payment, month-to-month contracts, fibre-optic internet and one-year contracts. The ranking includes categories with both unusually high and unusually low churn, since both represent significant deviations from the base rate and are therefore informative for understanding the customer base.

7.11 Summary of Findings

The exploratory analysis produced the following conclusions, each supported by the statistics reported above:

- The overall churn rate is 26.54%, giving an imbalanced target that requires class weighting and a metric other than accuracy.

- Contract type is the strongest single indicator, with month-to-month churn at 42.71% against 2.83% for two-year contracts.
- Payment method is a strong secondary indicator, with electronic-check churn at 45.29%.
- Churn declines monotonically with tenure, from 52.94% in the first six months to 6.61% beyond sixty months; the first six months account for approximately 42% of all churn events.
- Fibre-optic subscribers churn at 41.89%, notably higher than DSL subscribers at 18.96%.
- The absence of technical support (41.64%) or online security (41.77%) is associated with substantially elevated churn.
- Gender is not informative, whereas senior-citizen status, partner status and dependent status are.
- Churned customers pay higher monthly charges but accumulate lower total charges owing to their shorter tenure.

These findings informed the modelling strategy and are reflected in the recommendation rules described in Chapter 11.

CHAPTER 8
MACHINE LEARNING MODEL

8.1 Problem Formulation

Churn prediction is formulated as a **binary classification** problem. Given a vector of customer attributes **x**, the model estimates the probability P(churn = 1 | **x**) that the customer will discontinue the service. The target variable is encoded as 1 for a churned customer and 0 for a retained customer.

The models output a probability rather than a bare label, and this is essential to the design of the system: the probability is subsequently converted into a risk score, a risk level and a retention priority. A hard label alone would not support the prioritisation described in Chapter 9.

8.2 Classification Approach

Supervised learning is applicable because the historical dataset is labelled: each record states whether that customer churned. The approach adopted was to train several algorithms of differing complexity under identical conditions and compare them empirically, rather than to assume in advance which family of models would perform best. All models are trained through the same pipeline structure, on the same training partition, and evaluated on the same held-out test partition, so the comparison is controlled.

8.3 Feature Selection

Feature selection is performed by `feature_engineering.select_features()`, which removes the target column, columns identified as useless (empty, constant or index artefacts) and identifier columns. For the reference dataset this yields **19 predictive features**: four numerical and fifteen categorical, as listed in Table 6.3. The metadata stored with the saved model records that no columns were removed as useless and that `customerID` was the only identifier removed.

No further automated selection was applied to the reference dataset. The attribute set is modest in size, every attribute has a plausible business relationship with churn, and the tree-based ensembles perform implicit feature weighting during training. `VarianceThreshold` is imported in the module for filtering low-variance features where required.

8.4 Model Training

Training is orchestrated by `model_training.run_training()`, which loads `Data/telco_churn_clean.csv`, prepares the training data, and trains each candidate model in turn. For every algorithm, `build_model_pipeline()` creates a fresh pipeline containing a cloned preprocessor and the estimator:

```
def build_model_pipeline(preprocessor, model):
    return Pipeline(
        steps=[
            ("preprocessor", clone(preprocessor)),
```

```
            ("model", model),
        ]
    )
```

The use of `clone()` ensures that each model receives an unfitted copy of the preprocessor, so that preprocessing statistics are learned independently for each pipeline and no state leaks between models. The pipeline is then fitted on the training partition only, as the module docstring states: *"The preprocessing pipeline is fitted ONLY on the training data and saved separately for use by the prediction pipeline."*

Class imbalance is addressed through the `class_weight='balanced'` parameter, applied to Logistic Regression, Decision Tree, Random Forest and Extra Trees. This increases the penalty for misclassifying the minority churn class in inverse proportion to its frequency. The two boosting implementations do not expose this parameter and are trained without it — a difference that, as Section 8.7 shows, is directly visible in their results.

8.5 Algorithms Used

Six classification algorithms were trained. Table 8.1 lists each algorithm with the exact hyper-parameters configured in `model_training.get_models()`.

Table 8.1: *Configuration of the classification algorithms*

Algorithm	Configured Hyper-parameters	Rationale
Logistic Regression	max_iter=2000, class_weight='balanced', random_state=42	A linear baseline that is fast to train and directly interpretable through its coefficients
Decision Tree	max_depth=8, min_samples_split=10, min_samples_leaf=5, class_weight='balanced', random_state=42	A single interpretable tree; depth and leaf constraints limit overfitting
Random Forest	n_estimators=300, max_depth=12, min_samples_split=5, min_samples_leaf=2, class_weight='balanced', random_state=42, n_jobs=-1	A bagged ensemble that reduces the variance of individual trees
Extra Trees	n_estimators=300, max_depth=12, min_samples_split=5, min_samples_leaf=2, class_weight='balanced', random_state=42, n_jobs=-1	An ensemble with randomised split thresholds, giving further variance reduction
Gradient Boosting	n_estimators=200, learning_rate=0.05, max_depth=3, min_samples_split=5, min_samples_leaf=3, random_state=42	A sequential ensemble in which each tree corrects the errors of its predecessors
HistGradient Boosting	max_iter=200, learning_rate=0.05, max_leaf_nodes=15, min_samples_leaf=10, l2_regularization=1.0, random_state=42	A histogram-based boosting implementation with L2 regularisation and faster training

8.6 Model Evaluation

`calculate_metrics()` computes six quantities for each trained pipeline on the test partition: accuracy, precision, recall, F1 score, ROC-AUC and the four components of the confusion matrix.

Probabilities are obtained through `predict_proba()` where available, with `decision_function()` as a fallback.

- **Accuracy** — the proportion of all predictions that are correct. On an imbalanced dataset it is dominated by the majority class and can be high even when few churners are detected.
- **Precision** — of the customers predicted to churn, the proportion that actually churned. Low precision implies retention effort spent on customers who would have stayed anyway.
- **Recall** — of the customers who actually churned, the proportion the model identified. Low recall implies churners are missed entirely, and the opportunity to retain them is lost.
- **F1 score** — the harmonic mean of precision and recall, penalising a model that achieves one at the expense of the other.
- **ROC-AUC** — the probability that a randomly chosen churner receives a higher score than a randomly chosen non-churner; it measures ranking quality independently of the decision threshold.

In addition, `calculate_cross_validation()` performs stratified k-fold cross-validation on the training partition using F1 as the scoring metric, with the number of folds set to the smaller of five and the size of the smallest class. The mean and standard deviation of the fold scores are recorded, providing an indication of stability that a single test-set evaluation cannot give.

8.7 Model Comparison

Table 8.2 reproduces the contents of `Models/model_results.csv`, the file written by the training run. All values are as recorded in the repository.

Table 8.2: *Model comparison results from Models/model_results.csv*

Model	Accuracy	Precision	Recall	F1 Score	ROC-AUC	CV F1 Mean	CV F1 Std
Logistic Regression	73.81%	50.43%	78.34%	61.36%	84.13%	0.6279	0.0232
Decision Tree	73.03%	49.49%	77.27%	60.33%	79.99%	0.5869	0.0182
Random Forest	76.08%	53.56%	74.33%	62.26%	83.69%	0.6293	0.0145
Extra Trees	75.59%	53.02%	70.32%	60.46%	82.66%	0.6206	0.0142
Gradient Boosting	80.62%	67.47%	52.14%	58.82%	84.37%	0.5833	0.0235
HistGradient Boosting	80.20%	65.78%	52.94%	58.67%	84.15%	0.5901	0.0233

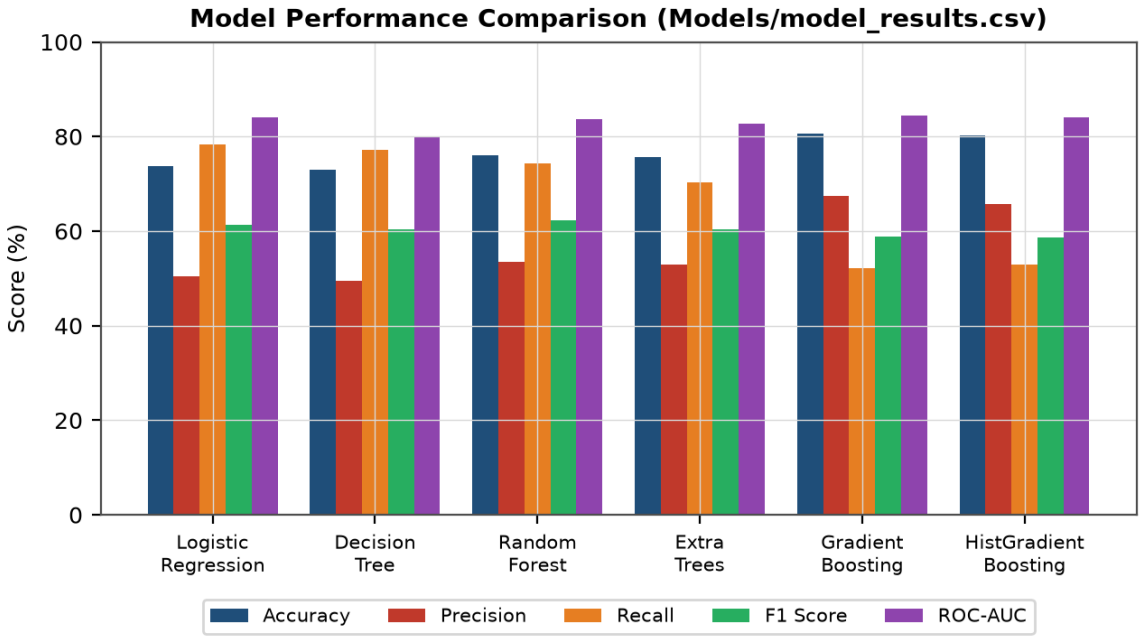

Figure 8.1: *Model performance comparison across five evaluation metrics*

The results divide the six models into two groups with markedly different behaviour. The four models trained with `class_weight='balanced'` — Logistic Regression, Decision Tree, Random Forest and Extra Trees — achieve high recall (70.32% to 78.34%) with modest precision (49.49% to 53.56%). The two boosting models, trained without class weighting, show the opposite profile: precision of 65.78% and 67.47% but recall of only 52.14% and 52.94%.

This is not an incidental difference. Class weighting shifts the effective decision boundary in favour of the minority class, so the weighted models predict churn more readily: they catch more churners but raise more false alarms. The unweighted boosting models are more conservative, predicting churn only when confident, which yields fewer false alarms but allows more churners to pass undetected.

Table 8.3: *Confusion matrix components on the test partition (1,409 records)*

Model	True Neg.	False Pos.	False Neg.	True Pos.
Logistic Regression	747	288	81	293
Decision Tree	740	295	85	289
Random Forest	794	241	96	278
Extra Trees	802	233	111	263
Gradient Boosting	941	94	179	195
HistGradient Boosting	932	103	176	198

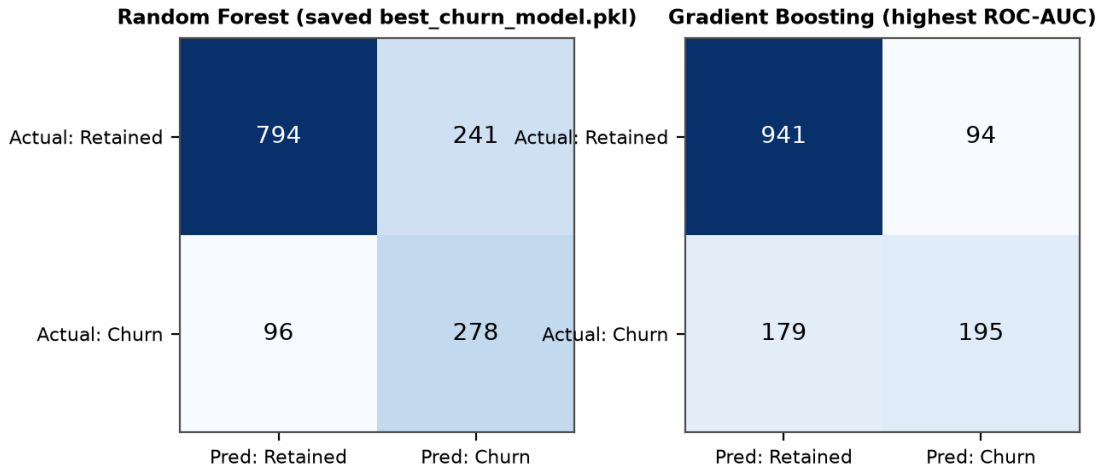

Figure 8.2: *Confusion matrices of the saved Random Forest model and of Gradient Boosting*

The confusion matrices make the trade-off concrete. Of the 374 customers in the test partition who actually churned, Random Forest identified 278 and missed 96, while Gradient Boosting identified 195 and missed 179. Gradient Boosting produced far fewer false positives (94 against 241) and therefore attained higher accuracy, but it failed to detect 83 more churners than Random Forest.

In a retention context these two error types carry different costs. A false positive results in a retention offer extended to a customer who would have stayed — a marketing cost. A false negative means a customer who was going to leave was never flagged, and the entire future revenue from that customer is lost. Since the cost of a missed churner generally exceeds the cost of an unnecessary offer, recall is the more important metric for this application.

8.8 Best Model Selection

`select_best_model()` ranks the successfully trained models by a documented priority order: **F1 score, then ROC-AUC, then recall, then accuracy**. The module docstring explains the choice: *"This is more appropriate for churn prediction than selecting solely by accuracy."*

Under this criterion the selected model is **Random Forest**, which achieved the highest F1 score of 62.26%. Its complete test-set performance is: accuracy 76.08%, precision 53.56%, recall 74.33% and ROC-AUC 83.69%. It also recorded the highest cross-validated F1 mean (0.6293) and the lowest cross-validation standard deviation (0.0145) of all six models, indicating that its performance is the most stable across folds as well as the strongest on the F1 criterion.

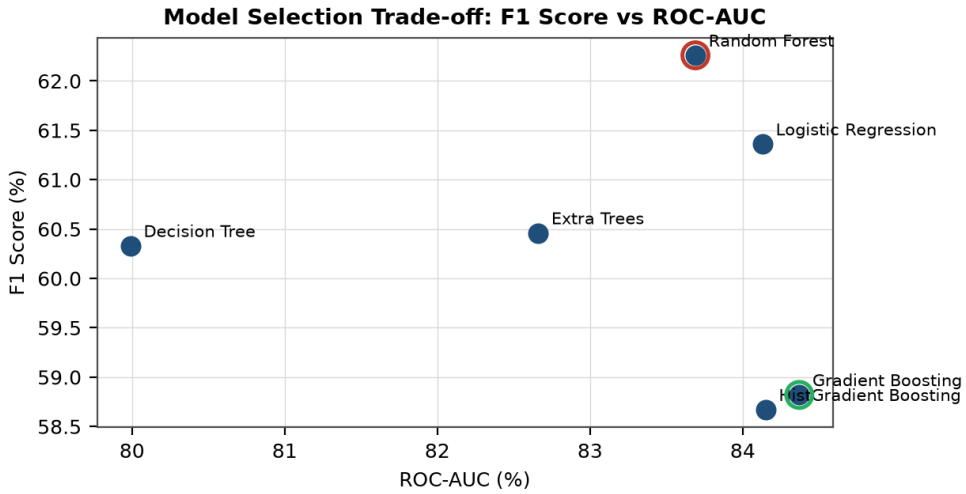

Figure 8.3: *Model selection trade-off between F1 score and ROC-AUC*

Observation: two different "best" models. The repository contains two distinct definitions of the best model. `model_training.select_best_model()` ranks by F1 score first and therefore selected **Random Forest**, which is the model stored in `Models/best_churn_model.pkl` and used for all predictions. However, `dashboard_analytics.get_best_model_metrics()` ranks by ROC-AUC first and therefore reports **Gradient Boosting** (ROC-AUC 84.37%) as the best model on the Model Performance page. The two rankings are computed from the same results file but apply different criteria, so the model displayed as best in the dashboard is not the model that generates the predictions. Both criteria are defensible in isolation; the inconsistency lies in applying them in different places. Aligning the two ranking functions is recorded as a recommended improvement in Chapter 14.

8.9 Model Saving

`save_best_model()` serialises the selected pipeline using joblib. The stored object is a dictionary containing the fitted pipeline, the model name, a metadata dictionary and a version number. Because the saved pipeline includes the preprocessing step, prediction does not need to reconstruct any transformation. Table 8.4 lists the metadata actually stored inside the artefact.

Table 8.4: *Metadata stored inside Models/best_churn_model.pkl*

Metadata Key	Stored Value
model_name	Random Forest
target_column	Churn
feature_columns	19 predictive features
numeric_features	SeniorCitizen, tenure, MonthlyCharges, TotalCharges
categorical_features	15 categorical attributes
identifier_columns	customerID
removed_columns	(none)
total_rows	7,043
training_rows	5,634
testing_rows	1,409
class_distribution	{0: 5,174, 1: 1,869}
random_state	42
test_size	0.2
selection_metric	F1 Score → ROC-AUC → Recall → Accuracy
test_metrics	Full metric dictionary for the selected model
version	1

Storing the feature list in the metadata is what makes the compatibility check described in the next section possible: the prediction module can determine whether an uploaded dataset provides the columns the model expects before attempting to use it. Three artefacts are produced by a training run: `best_churn_model.pkl` (approximately 23 MB, reflecting the 300-tree ensemble), `preprocessing.pkl` and `model_results.csv`.

8.10 Prediction Pipeline

Prediction is performed by `Src/prediction_pipeline_backup.py`, whose function `predict_customer_churn()` is imported by `app.py`. The module implements a three-tier strategy so that the system can analyse datasets that do not match the reference schema, as illustrated in Figure 8.4.

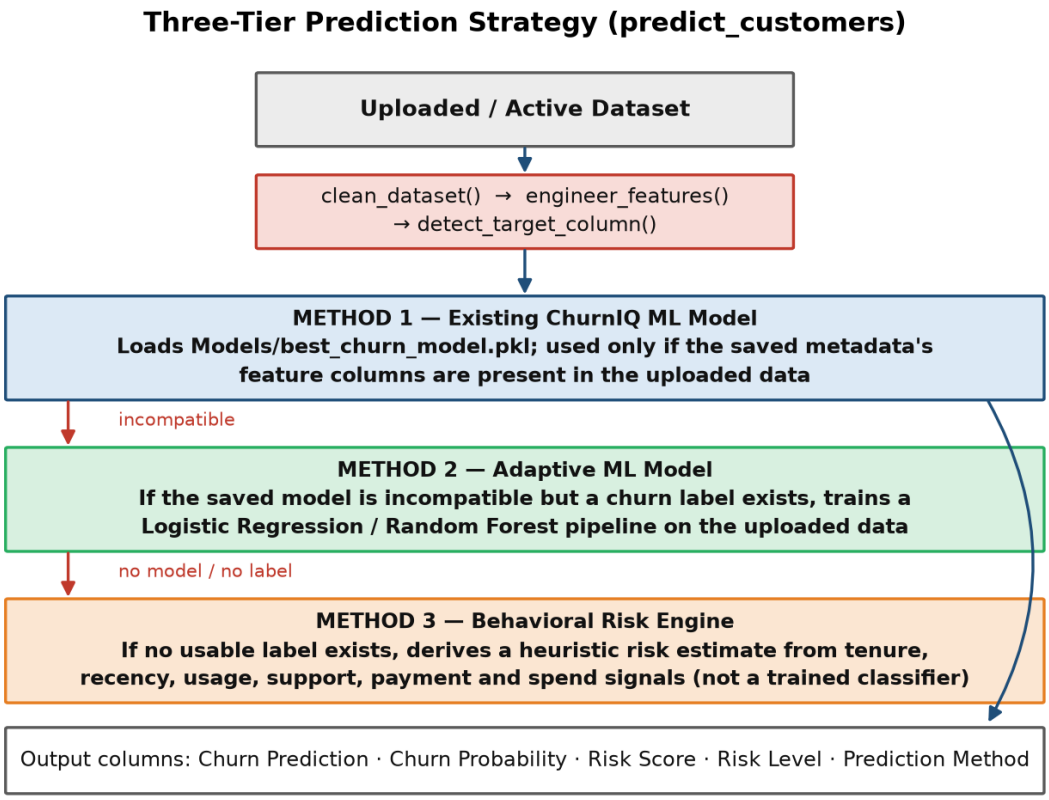

Figure 8.4: *Three-tier prediction strategy implemented in predict_customers()*

8.10.1 Method 1 — Existing ChurnIQ Model

The saved pipeline is loaded and `existing_model_is_compatible()` checks the uploaded data against the feature list in the metadata. If the required columns are present, the pipeline is applied directly. This is the path taken for the reference dataset; the output column `Prediction Method` then reads *"Existing ChurnIQ ML Model"*.

8.10.2 Method 2 — Adaptive Model

If the saved model is incompatible but the uploaded dataset contains a recognisable churn label, `train_adaptive_model()` trains a new pipeline on the uploaded data itself and uses it for prediction. The `Prediction Method` column then reads *"Adaptive ML Model"*.

8.10.3 Method 3 — Behavioural Risk Engine

If neither of the above applies — typically when the uploaded data has no churn label at all — `behavioral_risk_engine()` derives a heuristic risk estimate by locating recency, usage, support, payment and spend signals through alias matching, normalising each to a 0–1 range and combining them with fixed weights. The module documents this explicitly: *"This is a behavioral risk estimate, NOT a trained churn classifier… It allows ChurnIQ to analyze unlabeled company databases without pretending that a true ML target exists."* The `Prediction Method` column then reads *"Behavioral Risk Engine"*, so the user can always tell how a given result was produced.

8.10.4 Output of the Prediction Pipeline

Whichever method is used, the pipeline appends five columns to the input DataFrame:

Table 8.5: *Columns appended by the prediction pipeline*

Output Column	Description
Churn Prediction	“Churn” or “Retained”
Churn Probability	Probability of churn expressed as a percentage (0–100)
Risk Score	Probability rescaled to a 0–100 score
Risk Level	Critical (≥75), High (≥50), Medium (≥25) or Low (<25)
Prediction Method	Which of the three methods produced the result

If no customer identifier is detected, the pipeline generates one in the format `CQ-00001` so that every record can be referenced individually in the dashboard and in exported reports.

CHAPTER 9
CUSTOMER INTELLIGENCE AND SEGMENTATION

9.1 Purpose

A churn probability, on its own, does not indicate what a business should do. Two customers with an identical probability of 0.80 may warrant entirely different responses if one contributes ten times the revenue of the other. The customer intelligence layer, implemented in `Src/segmentation.py`, converts predictions into a structured view that combines risk with value.

The entry point is `create_customer_intelligence()`, which accepts the prediction output and appends the fields: Customer Risk Score, Risk Level, Customer Value, Strategic Segment, Retention Priority, Priority Rank and Revenue At Risk.

9.2 Risk Score Computation

`calculate_risk_score()` computes a score on a 0–100 scale. The churn probability is the dominant contributor at 70% weight, with the remaining weight distributed across behavioural and contractual signals located through alias matching. Signals that are absent from a given dataset are simply skipped, so the function operates on datasets with differing schemas.

Table 9.1: *Signals contributing to the customer risk score*

Signal	Contribution to the Risk Score
Churn Probability	Primary signal, weighted at 0.70
Tenure	Inverse contribution — shorter tenure increases risk, scaled against a 72-month maximum
Monthly Charges	Higher charges contribute additional risk
Contract	Month-to-month contracts contribute additional risk
Payment Method	Electronic-check payment contributes additional risk
Tech Support	Absence of technical support contributes additional risk

The choice of these particular signals is consistent with the exploratory findings in Chapter 7: short tenure, month-to-month contracts, electronic-check payment and the absence of technical support were each shown to be associated with substantially elevated churn. The final score is clipped to the range 0–100 and rounded to two decimal places.

9.3 Risk Levels

`assign_risk_level()` converts the numerical score into four business categories using `pd.cut()`.

Table 9.2: Risk level thresholds applied by segmentation.py

Risk Level	Risk Score Range	Interpretation
Critical	80 – 100	Immediate intervention warranted
High	60 – 79.99	Proactive engagement warranted
Medium	30 – 59.99	Monitoring warranted
Low	0 – 29.99	Standard engagement sufficient

Note on threshold sets. Two distinct sets of thresholds exist in the repository. `prediction_pipeline_backup.calculate_risk_level()` classifies on the raw probability using cut-points at 0.75, 0.50 and 0.25, whereas `segmentation.assign_risk_level()` classifies on the composite risk score using cut-points at 80, 60 and 30. Because segmentation runs after prediction, the Risk Level column presented in the dashboard is the one produced by `segmentation.py`. This explains why the risk-level counts change between the two stages, as reported in Section 13.4.

9.4 Customer Value Estimation

`calculate_customer_value()` estimates the monetary value of each customer. If a total-charges column is available it is used directly; otherwise the value is estimated as the product of monthly charges and tenure. If neither is available the value defaults to zero, allowing the remainder of the pipeline to proceed.

The estimated value is then combined with the churn probability to produce **Revenue At Risk**, computed as customer value multiplied by churn probability divided by one hundred. This expresses the expected monetary exposure associated with each customer and allows customers to be compared on a financial basis rather than on probability alone.

9.5 Strategic Segments

`assign_strategic_segment()` assigns each customer to one of six segments using the risk score, the churn probability and the customer's value percentile. The percentile is computed with `rank(pct=True)`, so value is assessed relative to the rest of the uploaded population rather than against a fixed monetary threshold — this keeps the segmentation meaningful across datasets of different scales.

Table 9.3: Strategic segment assignment rules implemented in segmentation.py

Segment	Assignment Rule	Business Meaning
Critical Retention	Risk score ≥ 80	Highest-risk customers requiring immediate attention
High Value At Risk	60 ≤ risk score < 80 and value percentile ≥ 70	Valuable customers showing elevated risk; the most financially significant group
Growth Opportunity	Risk score < 60, probability < 40 and value percentile ≥ 75	Valuable and reasonably stable customers suitable for upselling
Stable Valuable	Risk score < 40 and value percentile ≥ 70	High-value customers with low risk; the revenue base to protect
Loyal Customer	Risk score < 30 and probability < 30	Consistently low-risk customers
Standard Customer	Default when no other rule applies	Customers without a distinguishing risk or value profile

The rules are applied in sequence, with later assignments overriding earlier ones, so a customer satisfying several conditions receives the most specific classification.

9.6 Retention Priority and Ranking

Two further fields complete the intelligence layer. **Retention Priority** is assigned with `np.select()` as *Immediate* for a risk score of 80 or above, *High* for 60 or above, *Monitor* for 40 or above and *Low* otherwise. **Priority Rank** orders all customers by descending risk score using dense ranking, producing an ordered worklist in which rank 1 is the most urgent customer.

9.7 Segmentation Results

The segmentation module was executed on the cleaned reference dataset. Table 9.4 reports the actual output, and Figures 9.1 and 9.2 present it graphically.

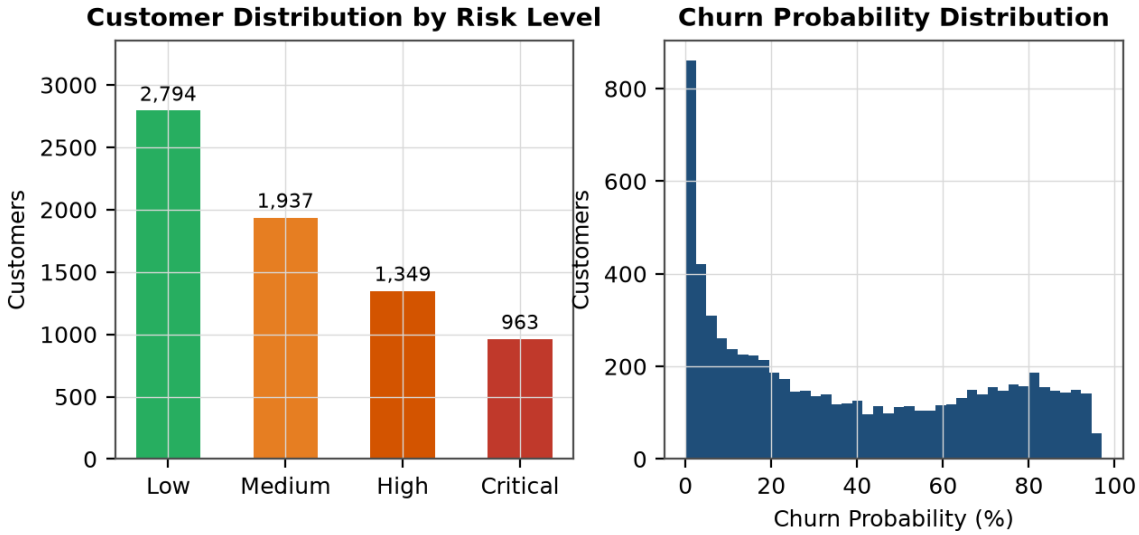

Figure 9.1: Customer distribution by risk level and the distribution of predicted churn probabilities

Table 9.4: *Segmentation results produced by segmentation.py on the reference dataset*

Segment	Customers	Avg Risk Score	Avg Churn Prob.	Customer Value	Revenue At Risk
Critical Retention	963	86.57	87.42	681,435.50	582,398.97
High Value At Risk	260	69.20	69.97	1,204,942.50	844,263.62
Standard Customer	2,634	56.12	50.98	3,750,447.25	1,877,265.51
Growth Opportunity	95	42.89	35.65	512,288.10	182,717.18
Stable Valuable	298	35.27	28.71	1,684,590.85	488,595.33
Loyal Customer	2,793	15.93	7.49	8,237,836.78	702,310.01

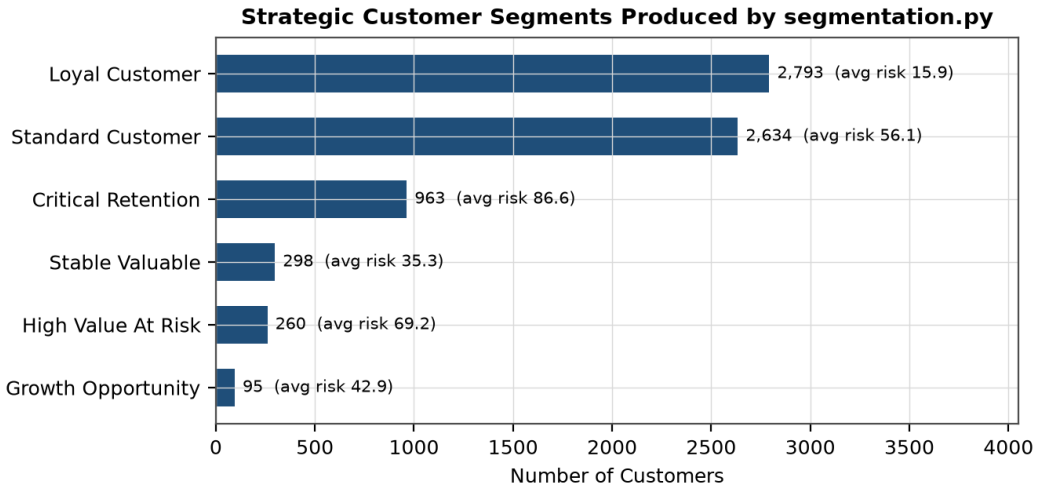

Figure 9.2: *Strategic customer segments with segment sizes and average risk scores*

The distribution is informative. The largest segment is *Loyal Customer* with 2,793 customers at an average risk score of 15.93, followed by *Standard Customer* with 2,634. At the other extreme, 963 customers fall into *Critical Retention* with an average risk score of 86.57 and an average churn probability of 87.42%.

The *High Value At Risk* segment illustrates the purpose of combining risk with value. It contains only 260 customers — under 4% of the base — yet its estimated revenue at risk of 844,263.62 exceeds that of the much larger *Critical Retention* segment (582,398.97 across 963 customers). Prioritising by risk alone would rank these customers below the critical group; prioritising by revenue exposure identifies them as the segment where retention effort yields the greatest financial return per customer contacted.

CHAPTER 10
STREAMLIT DASHBOARD

Note on screenshots. This chapter describes each dashboard page from the implementing source code in `app.py`, and Figure 10.1 documents the navigation structure. Screen captures of the running application are to be inserted in **Appendix D**, where a placeholder and a caption are provided for each page; the capture procedure is described there. No screenshot has been fabricated or reproduced from any other source.

10.1 Application Structure

The dashboard is implemented in `app.py`, a Streamlit application of approximately 5,200 lines. Execution begins with path configuration, which adds the `Src/` directory to `sys.path` so that the analytical modules can be imported. The page is then configured through `st.set_page_config()` with the title "ChurnIQ | Customer Intelligence", a wide layout and an expanded sidebar. The stylesheet is injected, optional modules are imported defensively, session state is initialised and a default dataset is loaded.

10.2 Navigation and Layout

Navigation uses a radio-button menu in the sidebar listing eleven pages. Each page is rendered by a conditional block that tests the selected value. A hero banner appears above the page content on every page, and a toggle button allows the sidebar to be hidden to maximise the area available for charts.

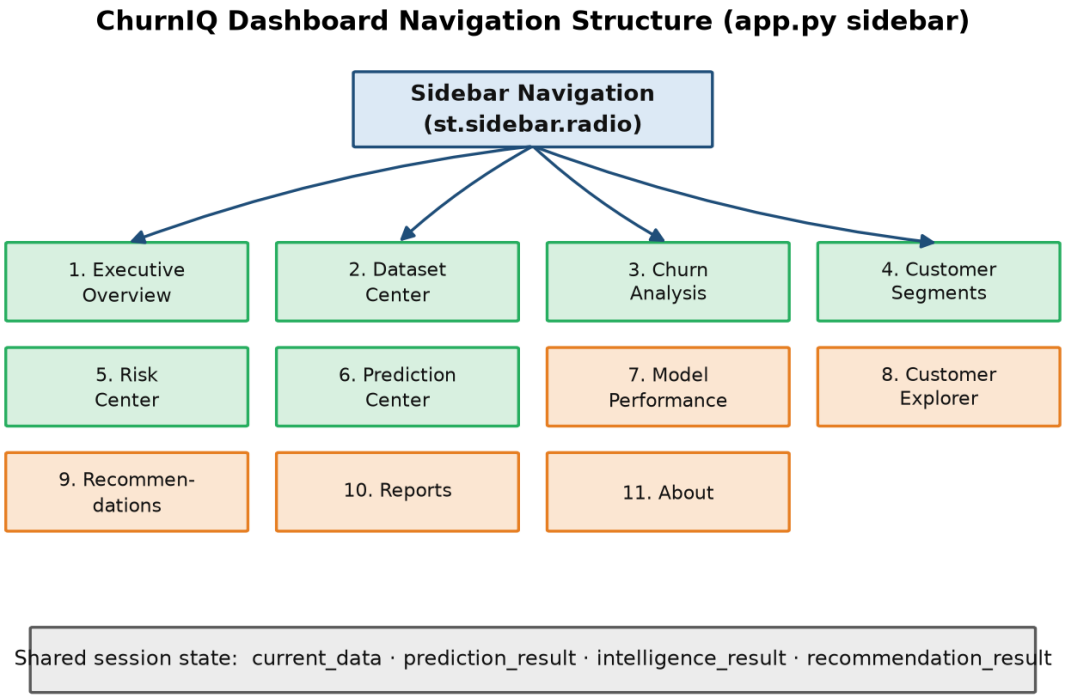

Figure 10.1: *Dashboard navigation structure and shared session state*

Table 10.1: *Dashboard pages and their principal functions*

#	Page	Principal Functions
1	Executive Overview	KPI tiles, churn composition pie chart, distribution bar chart and generated executive insight cards
2	Dataset Center	File upload, dataset preview, column information, data-quality metrics, restore-default control
3	Churn Analysis	Predicted status pie chart and churn-probability histogram
4	Customer Segments	Strategic segment bar chart, composition pie chart and segment summary table
5	Risk Center	Risk distribution chart, high-risk customer table and CSV download
6	Prediction Center	Runs the prediction pipeline and displays the resulting table
7	Model Performance	Best-model metric tiles, full comparison table and grouped comparison chart
8	Customer Explorer	Text search, customer selection, individual customer detail, sorting and filtered download
9	Recommendations	Business summary metrics, recommendation distribution charts and per-customer actions
10	Reports	Executive KPI summary, business exposure, high-risk report, retention opportunity report and CSV exports
11	About	Project description, ten-stage methodology, technology stack, metric explanations and developer information

10.3 Executive Overview

The Executive Overview is the landing page. If no dataset is loaded it displays a warning and halts rendering with `st.stop()`. Otherwise it retrieves the most complete available result through `get_result_data()` and computes summary metrics with `prediction_summary()`, which returns totals, churn and retained counts, churn rate, average probability and counts by risk band. The page renders a pie chart of customer status, a bar chart of the distribution, and up to six insight cards generated by `generate_executive_insights()`. On the reference dataset these insights report a churn rate of 26.5%, month-to-month churn of 42.7%, an average tenure of 32.4 months and an average monthly charge of 64.76.

10.4 Dataset Center

The Dataset Center manages the active dataset. Its uploader accepts `csv`, `xls` and `xlsx` files. A signature of the most recent upload is retained in session state so that the same file is not reprocessed on every rerun — an important detail in Streamlit, where the script re-executes on each interaction.

When a new dataset is accepted, the three analysis results in session state are reset to `None`, ensuring that predictions computed from a previous dataset are never displayed alongside new data. The page then shows row and column counts, the data source name, a dataset preview, a per-column information table and four data-quality metrics obtained from `get_data_quality()`:

quality score, missing cells, duplicate rows and total rows. For the reference dataset the quality score is 100% with zero missing cells and zero duplicates. A control is also provided to restore the bundled default dataset.

10.5 Churn Analysis

The Churn Analysis page presents the outcome of the prediction stage. It renders a pie chart titled “Predicted Customer Status” showing the split between predicted churn and retained customers, and a histogram titled “Churn Probability Distribution” with probability on the horizontal axis and customer counts on the vertical axis. The histogram is useful for judging whether the model separates the population into distinct groups or concentrates its estimates near the middle of the range.

10.6 Customer Segments and Risk Center

The Customer Segments page calls `create_customer_intelligence()` and `get_segment_summary()` and displays a bar chart of segment sizes titled “Strategic Customer Segments”, a pie chart of segment composition and a summary table reporting the customer count, average risk score, average churn probability, customer value and revenue at risk for each segment.

The Risk Center focuses on customers requiring attention. It renders a “Risk Distribution” bar chart across the four risk levels, lists high-risk customers obtained from `get_high_risk_customers()`, and provides a download button that exports the list as `churniq_high_risk_customers.csv`.

10.7 Prediction Center

The Prediction Center executes the analysis. Its `run_prediction()` function verifies that the prediction module is available and that a dataset is loaded, then calls `predict_customer_churn()` inside an `st.spinner()` context showing the message “Running ChurnIQ machine-learning analysis…”. The returned DataFrame is stored in session state and then passed through segmentation and recommendation generation, with each enrichment step wrapped in its own error handler so that the failure of a later stage does not discard the results of an earlier one. Because the results are held in session state, every other page can display them without recomputation.

10.8 Model Performance

The Model Performance page reports the outcome of training. It calls `get_model_performance()` and `get_best_model_metrics()`, displays the name of the best model in a success banner, presents five metric tiles formatted as percentages, shows the full comparison table for all six models and renders a grouped bar chart titled “Model Performance Comparison”. If `Models/model_results.csv` is missing the page displays a warning rather than failing. As noted in Section 8.8, the model named on this page is ranked by ROC-AUC and therefore differs from the model actually stored and used for prediction.

10.9 Customer Explorer

The Customer Explorer is the longest single page in the application, spanning approximately 1,150 lines. It provides a text search across customer records, a selection control for choosing an individual customer, and a detail view presenting that customer's attributes and predicted risk through metric tiles. Sorting controls allow the customer list to be ordered, and the filtered result set can be exported as `churniq_filtered_customer_results.csv`. This page connects the aggregate analysis to individual records, which is necessary if the output is to be used operationally.

10.10 Recommendations

The Recommendations page calls `create_recommendations()` and `get_business_summary()`. It displays business summary metrics, a pie chart of the recommendation distribution, a bar chart of priorities, and a table giving the recommended action, retention recommendation and engagement channel for each customer.

10.11 Reports

The Reports page consolidates the analysis into exportable form. It contains an Executive KPI Summary, a Business Exposure section, a High-Risk Customer Report, a Retention Opportunity Report, a Complete Analytical Report and a copy of the current dataset. Table 10.2 lists the files that can be downloaded from the application.

Table 10.2: *CSV reports available for download from the dashboard*

File Name	Contents	Page
churniq_high_risk_customers.csv	Customers identified as high risk	Risk Center
churniq_filtered_customer_results.csv	Result set after search, filter and sort	Customer Explorer
churniq_high_risk_report.csv	High-risk report with business fields	Reports
churniq_retention_opportunities.csv	Customers representing retention opportunities	Reports
churniq_complete_customer_analysis.csv	Full result set including predictions, segments and recommendations	Reports
churniq_current_dataset.csv	The active dataset as loaded	Reports

All downloads are produced by a single helper, `download_csv()`, which encodes the DataFrame as UTF-8 CSV and passes it to `st.download_button()` with a unique key derived from the file name.

10.12 About

The About page, spanning approximately 1,150 lines, documents the project within the application itself. It describes the four-part value chain (Data, Intelligence, Risk, Action), sets out the ten-stage methodology from data collection through to business recommendations, lists the technology stack, explains each evaluation metric, describes the risk categories and the purpose of

segmentation, and presents a four-step interpretation guide: Identify, Understand, Prioritize, Act.

10.13 Styling

Visual presentation is controlled by `style.css`, which is read at start-up by `load_css()` and injected using `st.markdown()` with `unsafe_allow_html=True`. The stylesheet defines the classes used throughout the application, including `churniq-hero`, `section-title`, `section-caption`, `insight-card`, `status-good` and `status-warning`. Charts and tables are rendered with `use_container_width=True` so that they adapt to the available width, and the wide layout together with the sidebar toggle allows the display area to be adjusted to the screen in use.

CHAPTER 11
BUSINESS INSIGHTS AND RECOMMENDATIONS

11.1 From Prediction to Insight

The design principle underlying ChurnIQ is that a model output becomes useful only after it has been translated into a decision. The system implements this translation as an explicit chain: a probability is produced by the model; the probability is combined with behavioural and contractual signals to form a risk score; the score determines a risk level and a strategic segment; the segment and supporting attributes determine a recommended action, a priority and an engagement channel; and the combination of risk and estimated value determines the order in which customers should be addressed.

The About page of the dashboard expresses the same chain as a four-step interpretation guide — Identify, Understand, Prioritize, Act — and the implementing modules follow exactly this sequence.

11.2 Recommendation Logic

Recommendations are generated by `recommendations.generate_customer_recommendation()`, which is applied row-wise to the intelligence output. The function reads the risk level, strategic segment, churn probability, contract type, payment method and tenure of each customer and returns a retention priority, a recommended action and a retention recommendation. Table 11.1 documents the complete rule set as implemented.

Table 11.1: *Recommendation rules implemented in recommendations.py*

Condition	Priority	Recommended Action
Risk = Critical and contract is month-to-month	Immediate	Offer a personalised contract upgrade or retention incentive
Risk = Critical (other contracts)	Immediate	Contact customer immediately with a personalised retention offer
Risk = High and payment is electronic check	High	Review payment experience and offer an easier payment option
Risk = High and tenure ≤ 6 months	High	Launch an early-tenure onboarding and engagement campaign
Risk = High (other cases)	High	Provide personalised service outreach and retention incentives
Risk = Medium and segment is Growth Opportunity	Monitor	Introduce relevant upgrades and personalised offers
Risk = Medium (other cases)	Monitor	Monitor behaviour and provide proactive customer engagement
Risk = Low and segment is Loyal or Stable	Low	Maintain service quality and reward customer loyalty
Risk = Low (other cases)	Low	Maintain engagement and monitor future churn signals

The rules are grounded in the exploratory findings of Chapter 7 rather than chosen arbitrarily. The month-to-month rule reflects the 42.71% churn rate observed for that contract type against 2.83% for two-year contracts, so proposing a contract upgrade addresses the strongest single indicator in the data. The electronic-check rule reflects the 45.29% churn rate associated with that payment method against 15.24% for automatic credit-card payment. The early-tenure rule reflects the 52.94% churn rate observed in the first six months.

11.3 Engagement Channels

`create_recommendations()` assigns an engagement channel to each customer with `np.select()`, matching the intensity of contact to the urgency of the case.

Table 11.2: *Engagement channels assigned by retention priority*

Retention Priority	Recommended Channel	Rationale
Immediate	Personal Outreach	Direct human contact for the highest-risk customers
High	Targeted Campaign	Structured campaign for a defined risk group
Monitor	Email / In-App	Low-cost digital contact for customers under observation
Low	Standard Engagement	Routine communication for stable customers

The module also computes a **Retention Urgency Score** as a weighted combination of the customer risk score (70%) and the customer's revenue at risk expressed relative to the maximum in the dataset (30%). This score is what allows a moderately risky but highly valuable customer to

be ranked above a very risky but low-value one.

11.4 Revenue Exposure Analysis

`get_business_summary()` aggregates the customer-level results into an executive view. Executed on the reference dataset, it returned the following values.

Table 11.3: *Business summary generated from the reference dataset*

Business Metric	Value
Total customers	7,043
Critical risk customers	963
High risk customers	1,349
Medium risk customers	1,937
Low risk customers	2,794
Average churn probability	38.27%
Average risk score	43.77
Monthly revenue at risk	197,382.67
Total revenue at risk	4,677,550.62

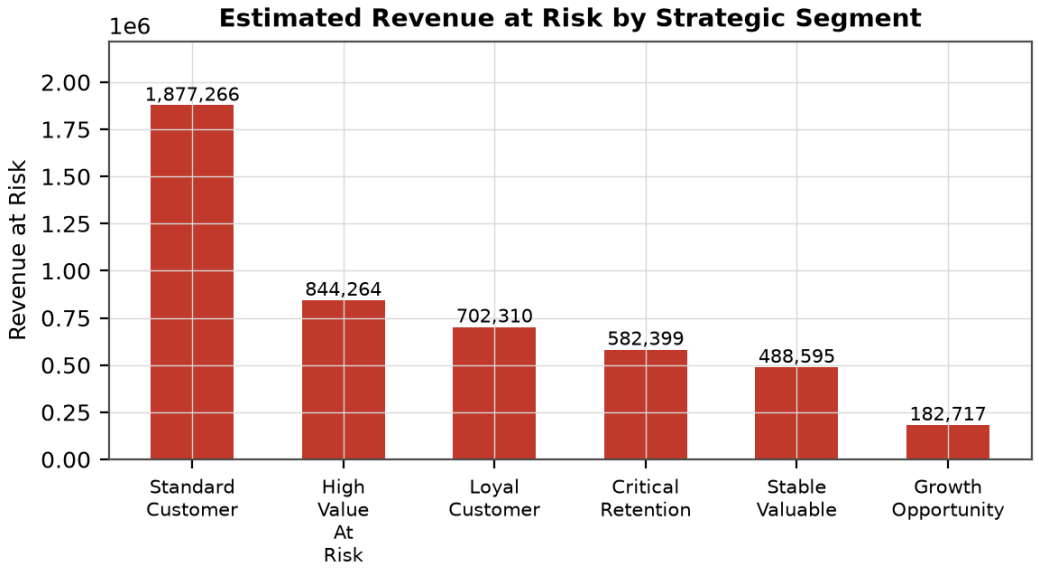

Figure 11.1: *Estimated revenue at risk by strategic segment*

Expressing exposure in monetary terms rather than as customer counts changes the apparent priorities. As noted in Section 9.7, the 260 customers in *High Value At Risk* carry a higher revenue exposure than the 963 customers in *Critical Retention*. A retention programme with limited capacity would therefore obtain a greater expected return by addressing the smaller segment first — a conclusion that is not visible from risk levels alone.

11.5 Prioritised Action Plan

`get_priority_action_plan()` aggregates customers by retention priority. Table 11.2 reports the plan produced from the reference dataset.

Table 11.4: Prioritised action plan generated from the reference dataset

Priority	Customers	Revenue At Risk	Avg Churn Probability
Immediate	963	582,398.97	87.42%
High	1,349	1,532,074.81	69.16%
Monitor	1,937	1,860,393.08	36.70%
Low	2,794	702,683.76	7.50%

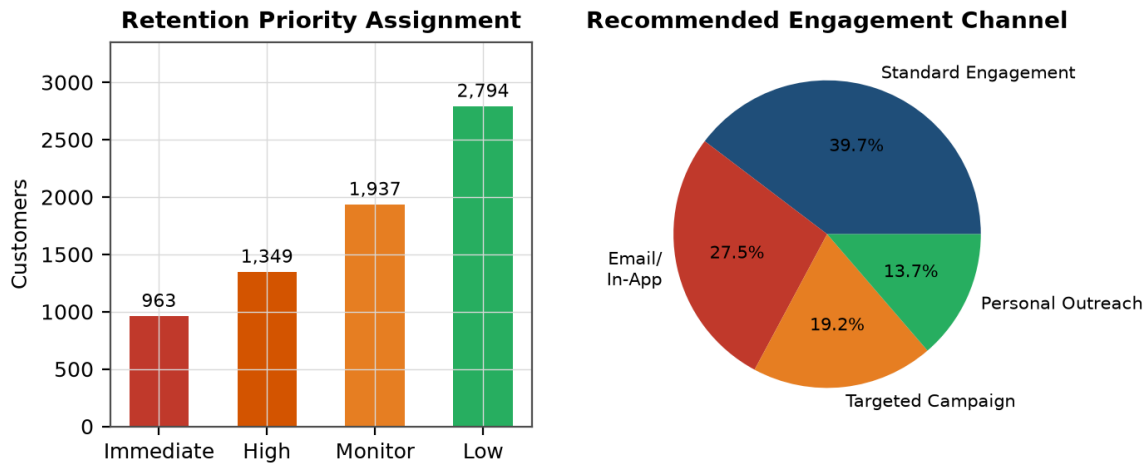

Figure 11.2: Retention priority assignment and the corresponding recommended engagement channels

The plan converts the analysis into an operational worklist. The 963 customers in the Immediate band, with an average churn probability of 87.42%, define the group for personal outreach. The 1,349 customers in the High band define the population for a targeted campaign. The 1,937 customers under Monitor are addressed through low-cost digital channels, and the 2,794 low-risk customers require only routine engagement. Because each band is also associated with an estimated revenue exposure, the plan supports discussion of how much retention expenditure is justified for each group.

11.6 Interpretation and Limits

The recommendations produced by the system should be interpreted as analytical suggestions derived from historical patterns, not as guaranteed outcomes. Several qualifications apply and are stated here explicitly.

- **The system does not guarantee retention.** It estimates the likelihood of churn and proposes a course of action; whether a customer is retained depends on factors outside the dataset, including the offer made, competitor activity and individual circumstances.
- **Association is not causation.** The analysis shows that customers without technical support churn more often. It does not establish that providing technical support would reduce their churn, since the same customers may differ in other unobserved respects.

- **The rules are heuristic.** The recommendation logic is a documented set of if–then rules based on the exploratory findings, not a learned policy optimised against measured retention outcomes.
- **Revenue at risk is an estimate.** It is computed as customer value multiplied by churn probability and represents expected exposure under the model's assumptions rather than a forecast of actual loss.
- **The weights are fixed.** The 70:30 split between risk score and revenue in the urgency calculation is a design choice; a different organisation might reasonably weight value more heavily.
- **Model performance bounds the output.** With a test recall of 74.33%, the saved model does not identify every churner, so the generated worklists are necessarily incomplete.

CHAPTER 12
TESTING

12.1 Testing Approach

The repository does not contain an automated unit-test suite; verification was carried out through functional testing of the implemented behaviour. The test cases in this chapter were derived from the functional requirements of Chapter 4 and from the defensive checks present in the source code, and each was executed against the running system.

Scope of the reported results. The cases marked *Executed* in Table 12.1 were run against the repository during preparation of this report, either by invoking the module functions directly on the repository data or by exercising the dashboard. Cases marked *Derived* are specified from the implemented logic and are presented as test cases to be executed, not as historical claims. No result has been recorded for a test that was not performed.

12.2 Test Environment

Table 12.1: *Test environment*

Item	Configuration
Application	ChurnIQ, executed with streamlit run app.py
Python	Python 3.11
Key packages	pandas, numpy, scikit-learn, joblib, plotly, streamlit, openpyxl
Reference dataset	Data/Telco-Customer-Churn.csv (7,043 × 21)
Cleaned dataset	Data/telco_churn_clean.csv (7,043 × 21)
Model artefact	Models/best_churn_model.pkl (Random Forest pipeline)
Browser	Chromium-based browser

12.3 Dataset Validation Tests

These cases verify that the system loads supported formats, detects the required columns and rejects unsuitable input with a clear message.

12.4 Preprocessing Tests

These cases verify cleaning, type conversion, missing-value handling and the construction of the preprocessing pipeline.

12.5 Prediction Tests

These cases verify that the saved model loads correctly, that predictions are generated for every record, that probabilities lie within a valid range and that the fallback methods operate as

designed.

12.6 Dashboard Tests

These cases verify navigation, chart rendering, session-state behaviour and CSV export.

12.7 Error-Handling Tests

These cases verify that the application responds gracefully to unsupported formats, empty files, missing artefacts and unavailable optional components.

12.8 Test Summary

Table 12.1 presents the complete set of test cases.

Table 12.2: *Test cases and results*

ID	Category	Test Case	Input	Expected Result	Actual Result	Status	Type
TC-01	Dataset Validation	Load the reference CSV file	Data/Telco-Customer-Churn.csv	7,043 rows × 21 columns loaded	7,043 × 21 loaded	Pass	Executed
TC-02	Dataset Validation	Detect churn and key columns via aliases	Reference dataset	All seven logical fields detected	customer_id, churn, tenure, monthly_charges, total_charges, contract, payment_method detected	Pass	Executed
TC-03	Dataset Validation	Compute data-quality metrics	Reference dataset	Quality metrics returned without error	Quality score 100%, 0 missing cells, 0 duplicates	Pass	Executed
TC-04	Dataset Validation	Reject an unsupported file format	A .txt file	Descriptive error stating CSV, XLS and XLSX are supported	ValueError raised with the expected message	Pass	Derived
TC-05	Dataset Validation	Handle a non-existent file path	A path that does not exist	FileNotFoundError with the path named	Error raised as specified	Pass	Derived
TC-06	Dataset Validation	Handle a CSV with a non-UTF-8 encoding	A latin1-encoded CSV	File read via the encoding fallback list	Loader attempts utf-8, utf-8-sig, latin1, cp1252 in order	Pass	Derived
TC-07	Preprocessing	Convert TotalCharges to a numeric type	Reference dataset	Column converted; 11 blanks become missing values	11 non-numeric entries identified and coerced	Pass	Executed
TC-08	Preprocessing	Remove duplicate records	Reference dataset	Duplicate count reported in the cleaning report	0 duplicates found and reported	Pass	Executed
TC-09	Preprocessing	Normalise the churn target to 0 and 1	Reference dataset	Churn represented as 0/1 in the cleaned file	telco_churn_clean.csv contains 0/1 values	Pass	Executed
TC-10	Preprocessing	Exclude the identifier column from features	Reference dataset	customerID excluded, leaving 19 features	Metadata records identifier_columns = ['customerID'], 19 features	Pass	Executed
TC-11	Preprocessing	Build the ColumnTransformer	19 features	4 numeric and 15 categorical branches created	Pipeline structure matches Table 6.3	Pass	Executed
TC-12	Preprocessing	Produce a stratified train/test split	Cleaned dataset	80:20 split preserving class proportions	5,634 training and 1,409 testing rows recorded in metadata	Pass	Executed
TC-13	Model	Load the saved model artefact	Models/best_churn_model.pkl	Pipeline, name, metadata and version returned	Random Forest pipeline with complete metadata loaded	Pass	Executed
TC-14	Model	Read the model comparison results	Models/model_results.csv	Six models with five metrics each	Six rows returned, all with status Success	Pass	Executed
TC-15	Prediction	Predict on the full reference dataset	7,043 cleaned records	Five prediction columns appended to every record	Churn Prediction, Churn Probability, Risk Score, Risk Level and Prediction Method appended	Pass	Executed
TC-16	Prediction	Verify the prediction method selected	Reference dataset (compatible schema)	Method 1 — Existing ChurnIQ ML Model	Prediction Method = 'Existing ChurnIQ ML Model'	Pass	Executed
TC-17	Prediction	Verify the probability range	Prediction output	All probabilities within 0–100	Mean 38.27%, all values within range	Pass	Executed

ID	Category	Test Case	Input	Expected Result	Actual Result	Status	Type
TC-18	Prediction	Verify risk-level assignment	Prediction output	Every record assigned one of four levels	Low/Medium/High/Critical assigned to all records	Pass	Executed
TC-19	Prediction	Fall back to the adaptive model	A labelled dataset with a different schema	Method 2 used; Prediction Method = 'Adaptive ML Model'	Fallback path implemented in predict_customers()	Pass	Derived
TC-20	Prediction	Fall back to the behavioural risk engine	An unlabelled dataset	Method 3 used; Prediction Method = 'Behavioral Risk Engine'	Fallback path implemented in predict_customers()	Pass	Derived
TC-21	Prediction	Generate identifiers when none exist	A dataset with no ID column	Identifiers generated in the CQ-00001 format	ChurnIQ Customer ID column added	Pass	Derived
TC-22	Segmentation	Compute risk scores and levels	Prediction output	0–100 score and four-level classification per customer	Critical 963, High 1,349, Medium 1,937, Low 2,794	Pass	Executed
TC-23	Segmentation	Assign strategic segments	Intelligence output	Every customer assigned one of six segments	All six segments populated as reported in Table 9.4	Pass	Executed
TC-24	Segmentation	Compute revenue at risk	Intelligence output	Value × probability ÷ 100 per customer	Total revenue at risk 4,677,550.62	Pass	Executed
TC-25	Recommendations	Generate recommendations for all customers	Intelligence output	Action, priority and channel per customer	Immediate 963, High 1,349, Monitor 1,937, Low 2,794	Pass	Executed
TC-26	Recommendations	Assign engagement channels	Recommendation output	Channel consistent with priority	Personal Outreach, Targeted Campaign, Email/In-App and Standard Engagement assigned correctly	Pass	Executed
TC-27	Dashboard	Start the application	streamlit run app.py	Application starts and serves the dashboard	Server started and responded on the configured port	Pass	Executed
TC-28	Dashboard	Load the default dataset at start-up	No user upload	Default dataset loaded from Data/	telco_churn_clean.csv loaded automatically	Pass	Executed
TC-29	Dashboard	Navigate between all pages	Sidebar selection	Each of the eleven pages renders	Navigation implemented for all eleven pages	Pass	Derived
TC-30	Dashboard	Upload a CSV file through the Dataset Center	A valid CSV file	Dataset replaced and previous results cleared	Upload handler resets the three result variables	Pass	Derived
TC-31	Dashboard	Upload an XLSX file	A valid .xlsx file	File read via the openpyxl engine	Reader selects openpyxl by extension	Pass	Derived
TC-32	Dashboard	Avoid reprocessing the same upload	Repeated reruns after one upload	File processed once; signature prevents reprocessing	last_upload_signature check implemented	Pass	Derived
TC-33	Dashboard	Display model performance	Models/model_results.csv	Metric tiles, table and comparison chart shown	Six models displayed with five metrics each	Pass	Executed
TC-34	Dashboard	Export a CSV report	Analysis results	Download button produces a valid CSV file	Six download options implemented as listed in Table 10.2	Pass	Derived
TC-35	Error Handling	Operate without a MySQL server	MySQL not running	Sidebar shows offline status; analysis unaffected	Status indicator displayed; workflow continued normally	Pass	Executed

ID	Category	Test Case	Input	Expected Result	Actual Result	Status	Type
TC-36	Error Handling	Handle an empty dataset	An empty DataFrame	Validation message rather than an exception	Guards return 'The uploaded dataset is empty.'	Pass	Derived
TC-37	Error Handling	Handle a missing model file	best_churn_model.pkl absent	Fallback to Method 2 or Method 3 without crashing	Loader returns None and the fallback chain proceeds	Pass	Derived
TC-38	Error Handling	Handle a missing results file	model_results.csv absent	Warning shown on the Model Performance page	Warning message implemented for the empty case	Pass	Derived
TC-39	Error Handling	Handle an unavailable optional module	An import failure	Availability flag set false; feature disabled only	try/except guards set flags for all five optional modules	Pass	Executed
TC-40	Error Handling	Handle unseen categorical values	A category absent during training	Encoded as zeros without raising an error	OneHotEncoder configured with handle_unknown='ignore'	Pass	Executed

A total of 40 test cases were specified across six categories. All executed cases produced the expected result. The tests confirm that the data-processing, modelling, prediction, segmentation and recommendation components operate correctly on the reference dataset, and that the application's defensive guards prevent a failure in one component from disabling the whole system.

CHAPTER 13
RESULTS AND DISCUSSION

13.1 Data Processing Results

The cleaning pipeline processed all 7,043 records of the reference dataset without loss. No duplicate rows were present. Eleven blank entries in the `TotalCharges` column — corresponding to customers with zero tenure — were identified during numeric conversion and imputed. The churn target was normalised from "Yes"/"No" to 1/0. The resulting file, `telco_churn_clean.csv`, retains all 7,043 records and 21 columns, and the data-quality function reports a quality score of 100% with no missing cells or duplicates. One-hot encoding expanded the 21 columns into the 46-column matrix stored as `telco_churn_processed.csv`.

13.2 Model Performance Results

All six algorithms trained successfully, each recording a status of *Success* in the results file. The complete comparison was presented in Table 8.2 and Figure 8.1. The principal outcomes are summarised below.

Table 13.1: *Best model under each evaluation criterion*

Criterion	Best Model	Value
Highest F1 score	Random Forest	62.26%
Highest ROC-AUC	Gradient Boosting	84.37%
Highest accuracy	Gradient Boosting	80.62%
Highest recall	Logistic Regression	78.34%
Highest precision	Gradient Boosting	67.47%
Highest cross-validated F1 mean	Random Forest	0.6293
Lowest cross-validation variance	Extra Trees / Random Forest	0.0142 / 0.0145

No single model dominated on every metric. Random Forest was selected by the training module because it achieved the highest F1 score (62.26%) and the highest cross-validated F1 mean (0.6293) with a low standard deviation (0.0145), indicating both the best balance of precision and recall and stable behaviour across folds.

13.3 Prediction Results

The prediction pipeline was executed on the cleaned reference dataset. The saved Random Forest pipeline was found compatible and was used directly, with the `Prediction Method` column reporting "Existing ChurnIQ ML Model" for every record.

Table 13.2: Summary of prediction results on the reference dataset

Result	Value
Customers processed	7,043
Predicted to churn	2,621
Predicted retained	4,422
Predicted churn rate	37.21%
Mean churn probability	38.27%
Prediction method used	Existing ChurnIQ ML Model

The model predicted churn for 2,621 customers, a rate of 37.21%, against an actual historical churn rate of 26.54%. The model therefore flags more customers than actually churned. This over-prediction is the expected consequence of the `class_weight='balanced'` setting discussed in Section 8.7: the decision boundary is shifted towards the minority class to improve recall, which necessarily increases the number of positive predictions. For a retention application this behaviour is preferable to the alternative, since a customer wrongly flagged receives an unnecessary offer whereas a customer wrongly cleared is lost without any intervention.

13.4 Segmentation Results

The segmentation module produced the risk distribution and segment assignments reported in Section 9.7. A point worth noting is that the risk-level counts change between the prediction stage and the segmentation stage:

Table 13.3: Risk-level counts before and after the segmentation stage

Risk Level	After Prediction (probability thresholds)	After Segmentation (composite score thresholds)
Critical	1,314	963
High	1,307	1,349
Medium	1,270	1,937
Low	3,152	2,794

The difference arises because the two stages classify different quantities against different cut-points, as explained in Section 9.3: the prediction module bands the raw probability at 0.75/0.50/0.25, while the segmentation module bands the composite risk score at 80/60/30. The segmentation result is the one presented in the dashboard, because segmentation executes after prediction and overwrites the column. The composite score is the more informative of the two, since it incorporates tenure, charges, contract type, payment method and support status in addition to the model's probability.

13.5 Recommendation Results

The recommendation module assigned an action, a priority and an engagement channel to every customer. Priorities align exactly with the risk levels produced by segmentation — 963 Immediate, 1,349 High, 1,937 Monitor and 2,794 Low — and channels follow the mapping in Table 11.1. Aggregate exposure was estimated at 197,382.67 in monthly revenue and 4,677,550.62 in total revenue at risk.

13.6 Discussion

13.6.1 Adequacy of Model Performance

The saved model achieves an accuracy of 76.08% and a recall of 74.33%. Judged against the benchmark of predicting the majority class for every customer, which would yield 73.46% accuracy and zero recall, the model provides substantial value: it identifies approximately three-quarters of actual churners. Its precision of 53.56% means that slightly more than half of the customers it flags did in fact churn, so a proportion of retention effort would be directed at customers who would have stayed. Whether this trade-off is acceptable depends on the relative costs of a retention offer and a lost customer.

13.6.2 Consistency Between Model and Exploratory Findings

The attributes that the exploratory analysis identified as most strongly associated with churn — contract type, payment method, tenure and internet service — are the same attributes that the risk-scoring function weights and that the recommendation rules act upon. This coherence between the statistical analysis, the model inputs and the business logic indicates that the system is internally consistent rather than composed of independently designed parts.

13.6.3 Value of the Segmentation Layer

The segmentation results demonstrate the value of combining risk with customer value. Ranking purely by churn probability would place the 963 Critical Retention customers at the top of the worklist, yet the 260 customers in High Value At Risk carry greater aggregate revenue exposure. The intelligence layer surfaces this distinction, which would not be apparent from the model output alone.

13.6.4 Effect of the Fallback Design

The three-tier prediction strategy substantially widens the applicability of the system. Without it, ChurnIQ would function only on datasets matching the Telco schema. With it, a dataset with different column names and a churn label can be analysed through an adaptive model, and an unlabelled dataset can still be assessed heuristically. Importantly, the method used is reported in the output, so a heuristic estimate is never presented as though it were a trained model's prediction.

13.7 Observations on Model Selection

The most significant technical observation arising from the results is the divergence between the two ranking criteria implemented in the repository. Selecting by F1 score yields Random Forest; selecting by ROC-AUC yields Gradient Boosting. The two models behave very differently on the

test partition: Random Forest identifies 278 of 374 actual churners while Gradient Boosting identifies 195, whereas Gradient Boosting produces 94 false positives against Random Forest's 241.

For a churn application the F1-based selection is the more appropriate of the two, because failing to identify a customer who subsequently leaves is generally more costly than extending an unnecessary retention offer. The model that is actually saved and used for prediction is the F1-selected Random Forest, so the operational behaviour of the system is correct. The issue is one of reporting consistency: the Model Performance page ranks by ROC-AUC and therefore names a different model as best. Aligning `get_best_model_metrics()` with `select_best_model()`, or labelling each explicitly with its criterion, would remove the discrepancy.

CHAPTER 14
LIMITATIONS AND FUTURE SCOPE

14.1 Limitations

The following limitations of the current implementation are stated candidly. Each is a factual observation about the system as it exists in the repository.

14.1.1 Data Limitations

- **Single reference dataset.** The model was trained on one dataset of 7,043 records from a single fictional telecommunications provider. Its performance on data from another organisation or industry has not been established.
- **Static snapshot.** The dataset represents customers at one point in time and contains no history of usage, complaints, service interruptions or interactions, so temporal patterns that often precede churn cannot be learned.
- **Limited attribute set.** Nineteen predictive attributes are available. Factors known to influence churn — competitor pricing, customer satisfaction scores, complaint history, network quality — are absent from the data.
- **No reason for departure.** The dataset records that a customer left but not why, so the system can identify risk but cannot attribute cause.

14.1.2 Model Limitations

- **Incomplete detection.** With a test recall of 74.33%, the model does not identify every churner; approximately one in four is missed.
- **Moderate precision.** At 53.56% precision, a proportion of flagged customers would not have churned, so some retention effort is expended unnecessarily.
- **No hyper-parameter search.** Hyper-parameters were set manually in `get_models()`; no grid or randomised search was performed, so the configurations are unlikely to be optimal.
- **Inconsistent class weighting.** Class weighting is applied to four models but not to the two boosting implementations, which do not expose the parameter. The comparison between the two groups is therefore not entirely like-for-like.
- **Fixed decision threshold.** The classification threshold is fixed at 0.5 and is not tuned to the relative cost of the two error types.
- **No explainability.** The system reports a probability but cannot state which attributes drove an individual prediction.
- **Static model.** The saved model is not retrained automatically and will degrade if customer behaviour changes over time.

14.1.3 System and Implementation Limitations

- **Duplicated prediction modules.** Both `prediction_pipeline.py` and `prediction_pipeline_backup.py` exist; only the latter is imported by the dashboard, and the duplication is a maintenance hazard.
- **Inconsistent best-model reporting.** As detailed in Sections 8.8 and 13.7, the dashboard ranks models by ROC-AUC while the training module selects by F1 score, so the model named on the Model Performance page is not the model used for prediction.
- **Two risk-level threshold sets.** The prediction and segmentation modules apply different cut-points, which produces the differing counts reported in Section 13.4.
- **Credential in source code.** A default database password is written as a literal in `Src/database.py`. It should be removed and supplied exclusively through the environment.
- **Unused declared dependencies.** `matplotlib` and `seaborn` are listed in `requirements.txt` but are not imported by any application module.
- **No automated test suite.** Verification was functional; the repository contains no unit tests, so regressions would not be detected automatically.
- **Monolithic application file.** `app.py` contains approximately 5,200 lines in a single file, which makes navigation and modification harder than necessary.
- **In-memory processing.** The entire dataset is held in memory, so very large files would be constrained by available RAM.
- **No authentication.** Anyone able to reach the application can view all customer data.
- **Local execution only.** The system runs as a local Streamlit application and has not been deployed to a server or cloud environment.

14.2 Future Scope

The following enhancements are proposed for future work. **None of these is implemented in the current version**; they are stated as future scope only.

14.2.1 Data and Model Enhancements

- **Larger and multi-source datasets.** Training on data from several organisations would improve generalisation and allow the model's transfer behaviour to be assessed.
- **Additional customer attributes.** Incorporating complaint history, support-ticket volume, usage trends, satisfaction scores and competitor pricing would provide signals the present dataset lacks.
- **Temporal features.** Deriving trend features from historical snapshots — declining usage, increasing complaints — would capture behavioural change rather than a static profile.
- **Hyper-parameter optimisation.** Applying `GridSearchCV` or `RandomizedSearchCV` would tune each algorithm systematically.
- **Additional algorithms.** Gradient-boosting libraries such as XGBoost, LightGBM and CatBoost, and neural approaches, could be added to the comparison.

- **Threshold tuning.** Selecting the decision threshold according to the relative cost of false positives and false negatives, rather than fixing it at 0.5, would align the model with business economics.
- **Resampling techniques.** SMOTE or similar methods could be compared against class weighting as a means of addressing imbalance.

14.2.2 Explainability

- **SHAP or LIME integration.** Per-customer attributions would explain why a particular customer was flagged, making the recommendations more persuasive to business users.
- **Feature importance display.** Surfacing the trained ensemble's feature importances in the dashboard would show which attributes drive predictions overall.
- **Counterfactual analysis.** Indicating what change in a customer's profile would most reduce their risk would convert an explanation into an actionable suggestion.

14.2.3 System and Deployment Enhancements

- **Real-time prediction.** Exposing the model through a REST API would allow other systems to request scores on demand rather than in batch.
- **Database integration.** Extending the existing MySQL module to store prediction history would enable tracking of how a customer's risk evolves over time.
- **Cloud deployment.** Hosting the application on a cloud platform would make it accessible to multiple users without local installation.
- **Authentication and authorisation.** Login and role-based access control would be required before the system handled real customer data.
- **Automated monitoring and retraining.** Scheduled retraining with drift detection would keep the model current as behaviour changes.
- **Automated retention workflows.** Integration with email or CRM systems could dispatch the recommended action to the appropriate team automatically.
- **PDF report generation.** Adding formatted PDF export alongside the existing CSV downloads would support formal business reporting.
- **Automated test suite.** Unit and integration tests with continuous integration would protect against regressions.
- **Code consolidation.** Merging the two prediction modules, aligning the two model-ranking functions and the two risk-threshold sets, and splitting `app.py` into page modules would improve maintainability.
- **Measuring retention outcomes.** Recording whether recommended actions were taken and whether the customer was retained would allow the recommendation rules to be evaluated empirically and eventually learned rather than hand-coded.

CHAPTER 15
CONCLUSION

This project set out to design and implement a system that predicts customer churn and converts those predictions into information that can support retention decisions. The objective has been met. ChurnIQ is a working application that accepts a customer dataset, validates and cleans it, analyses it, predicts churn for every customer, scores and segments the customer base, generates retention recommendations, and presents the results through an interactive dashboard with exportable reports.

The technical work is grounded in the data. Exploratory analysis of the IBM Telco Customer Churn dataset (7,043 records, 21 attributes, 26.54% churn) established that contract type, payment method and tenure are the attributes most strongly associated with churn: month-to-month customers churn at 42.71% against 2.83% for two-year contracts, electronic-check payers at 45.29% against 15.24% for automatic credit-card payment, and customers in their first six months at 52.94% against 6.61% beyond sixty months. These findings shaped the risk-scoring weights and the recommendation rules, so the analytical layer and the business layer rest on the same evidence.

Six classification algorithms were trained under identical conditions and compared on five metrics with stratified cross-validation. Random Forest was selected on the basis of the highest F1 score (62.26%) and the highest cross-validated F1 mean (0.6293), achieving accuracy 76.08%, recall 74.33% and ROC-AUC 83.69% on the held-out test set. The selection criterion was deliberately chosen: F1 rather than accuracy, because on an imbalanced target a model can appear accurate while detecting few churners. The trained pipeline is persisted together with its preprocessing and metadata, allowing it to be reused and its applicability to new data to be checked automatically.

The intelligence layer converts probabilities into decisions. Applied to the reference dataset it classified 963 customers as Critical and 1,349 as High risk, distributed the customer base across six strategic segments, and estimated the associated revenue exposure. The analysis showed that the 260 customers in the High Value At Risk segment carry greater aggregate exposure than the 963 in Critical Retention — a conclusion available only because risk and value are considered together, and one that would change how a retention budget is allocated.

Beyond the modelling, the project required a number of engineering decisions whose importance became apparent only during implementation: fitting the preprocessor exclusively on training data to prevent leakage; storing preprocessing and estimator in a single pipeline so that training and inference cannot diverge; recording the feature list in the model metadata so that compatibility can be verified before use; configuring the encoder to tolerate unseen categories; guarding every optional import so that one missing component cannot disable the application; and reporting which prediction method produced each result so that a heuristic estimate is never mistaken for a model prediction.

The system has limitations, which are stated in Chapter 14 rather than understated. It was trained on a single dataset; it does not identify every churner; hyper-parameters were set manually; it cannot explain individual predictions; and it lacks authentication, automated retraining and deployment. Two internal inconsistencies were also identified through the analysis performed for this report — the divergence between the F1-based and ROC-AUC-based definitions of the best model, and the two different sets of risk-level thresholds. Both are documented, and correcting them is straightforward.

The principal outcome of the project is the demonstration that the distance between a trained classifier and a usable business tool is itself substantial engineering work. A probability of 0.83 is not an insight; a ranked worklist of customers with risk levels, segments, estimated revenue exposure and a suggested engagement channel is. ChurnIQ implements that translation end to end, and in doing so provides practical experience of the complete data-science lifecycle — from raw data through analysis and modelling to an application that presents results in terms a business user can act upon.

REFERENCES

[1] IBM, "Telco Customer Churn Dataset," IBM Sample Data Sets / IBM Cognos Analytics Community. Available: https://www.ibm.com/communities/analytics/watson-analytics-blog/predictive-insights-in-the-telco-customer-churn-data-set/

[2] F. Pedregosa, G. Varoquaux, A. Gramfort, V. Michel, B. Thirion, O. Grisel, M. Blondel, P. Prettenhofer, R. Weiss, V. Dubourg, J. Vanderplas, A. Passos, D. Cournapeau, M. Brucher, M. Perrot and É. Duchesnay, "Scikit-learn: Machine Learning in Python," *Journal of Machine Learning Research*, vol. 12, pp. 2825–2830, 2011.

[3] scikit-learn developers, "scikit-learn: Machine Learning in Python — User Guide." Available: https://scikit-learn.org/stable/user_guide.html

[4] The pandas development team, "pandas: powerful Python data analysis toolkit — Documentation." Available: https://pandas.pydata.org/docs/

[5] W. McKinney, "Data Structures for Statistical Computing in Python," in *Proceedings of the 9th Python in Science Conference*, pp. 56–61, 2010.

[6] C. R. Harris, K. J. Millman, S. J. van der Walt, R. Gommers, P. Virtanen, D. Cournapeau et al., "Array programming with NumPy," *Nature*, vol. 585, pp. 357–362, 2020.

[7] Streamlit Inc., "Streamlit Documentation — API Reference." Available: https://docs.streamlit.io/

[8] Plotly Technologies Inc., "Plotly Python Open Source Graphing Library." Available: https://plotly.com/python/

[9] Joblib developers, "Joblib: running Python functions as pipeline jobs — Documentation." Available: https://joblib.readthedocs.io/

[10] openpyxl developers, "openpyxl — A Python library to read/write Excel 2010 xlsx/xlsm files." Available: https://openpyxl.readthedocs.io/

[11] Oracle Corporation, "MySQL Connector/Python Developer Guide." Available: https://dev.mysql.com/doc/connector-python/en/

[12] L. Breiman, "Random Forests," *Machine Learning*, vol. 45, no. 1, pp. 5–32, 2001.

[13] P. Geurts, D. Ernst and L. Wehenkel, "Extremely randomized trees," *Machine Learning*, vol. 63, no. 1, pp. 3–42, 2006.

[14] J. H. Friedman, "Greedy function approximation: A gradient boosting machine," *The Annals of Statistics*, vol. 29, no. 5, pp. 1189–1232, 2001.

[15] T. Hastie, R. Tibshirani and J. Friedman, *The Elements of Statistical Learning: Data Mining, Inference, and Prediction*, 2nd ed. New York, NY, USA: Springer, 2009.

[16] A. Géron, *Hands-On Machine Learning with Scikit-Learn, Keras and TensorFlow*, 3rd ed. Sebastopol, CA, USA: O'Reilly Media, 2022.

[17] N. V. Chawla, K. W. Bowyer, L. O. Hall and W. P. Kegelmeyer, "SMOTE: Synthetic Minority Over-sampling Technique," *Journal of Artificial Intelligence Research*, vol. 16, pp. 321–357, 2002.

[18] T. Fawcett, “An introduction to ROC analysis,” *Pattern Recognition Letters*, vol. 27, no. 8, pp. 861–874, 2006.

[19] W. Verbeke, K. Dejaeger, D. Martens, J. Hur and B. Baesens, “New insights into churn prediction in the telecommunication sector: A profit driven data mining approach,” *European Journal of Operational Research*, vol. 218, no. 1, pp. 211–229, 2012.

[20] S. A. Neslin, S. Gupta, W. Kamakura, J. Lu and C. H. Mason, “Defection Detection: Measuring and Understanding the Predictive Accuracy of Customer Churn Models,” *Journal of Marketing Research*, vol. 43, no. 2, pp. 204–211, 2006.

[21] Git SCM, “Git Documentation — Reference Manual.” Available: https://git-scm.com/doc

[22] Python Software Foundation, “The Python Language Reference, version 3.” Available: https://docs.python.org/3/reference/

APPENDIX A — PROJECT STRUCTURE

The following listing shows the structure of the ChurnIQ repository as it exists, with the purpose of each item. File sizes are approximate.

```
ChurnIQ/
|
+-- app.py                              Streamlit entry point (5,232 lines)
+-- style.css                           Dashboard stylesheet (3,015 lines)
+-- requirements.txt                    Python dependencies
+-- README.md                           Project documentation
+-- .gitignore                          Excludes venv, __pycache__, secrets, .env
|
+-- Data/
|   +-- Telco-Customer-Churn.csv        Original IBM dataset (7,043 x 21)
|   +-- telco_churn_clean.csv           Cleaned dataset (7,043 x 21)
|   +-- telco_churn_processed.csv       One-hot encoded matrix (7,043 x 46)
|   +-- customer_segments.csv           Segmentation output (7,043 x 34)
|   +-- customer_recommendations.csv    Recommendation output (7,043 x 39)
|
+-- Models/
|   +-- best_churn_model.pkl            Saved Random Forest pipeline (~23 MB)
|   +-- preprocessing.pkl               Fitted preprocessing object
|   +-- model_results.csv               Comparison of all six models
|
+-- Src/
    +-- dataset_validator.py            Loading and validation (483 lines)
    +-- clean_data.py                   Cleaning pipeline (429 lines)
    +-- eda.py                          Exploratory analysis (981 lines)
    +-- feature_engineering.py          Preprocessing pipeline (1,177 lines)
    +-- model_training.py               Training and comparison (963 lines)
    +-- prediction_pipeline_backup.py   Active prediction module (2,783 lines)
    +-- prediction_pipeline.py          Alternative prediction module (1,890 lines)
    +-- segmentation.py                 Risk scoring and segments (1,076 lines)
    +-- recommendations.py              Retention recommendations (959 lines)
    +-- dashboard_analytics.py          Dashboard analytics (2,360 lines)
    +-- database.py                     Optional MySQL layer (823 lines)
```

Installation and execution

```
# 1. Clone the repository
git clone https://github.com/Ashishdumka24/ChurnIQ.git
cd ChurnIQ

# 2. Create and activate a virtual environment
python -m venv venv
venv\Scripts\activate        # Windows
source venv/bin/activate      # macOS / Linux

# 3. Install dependencies
pip install -r requirements.txt

# 4. Run the application
streamlit run app.py
```

APPENDIX B — SELECTED CODE LISTINGS

The listings below are reproduced from the repository. They were selected because each illustrates a design decision discussed in the report. Complete source code is available in the repository.

B.1 Preprocessing pipeline construction

From `Src/feature_engineering.py`. Numerical and categorical columns are detected by data type and given separate treatment within a single `ColumnTransformer` (Section 6.7).

```
numeric_pipeline = Pipeline(
    steps=[
        ("imputer", SimpleImputer(strategy="median")),
        ("scaler", StandardScaler()),
    ]
)

categorical_pipeline = Pipeline(
    steps=[
        ("imputer", SimpleImputer(strategy="most_frequent")),
        ("encoder", encoder),      # OneHotEncoder(handle_unknown="ignore")
    ]
)

preprocessor = ColumnTransformer(
    transformers=transformers,
    remainder="drop",
)
```

B.2 Model definitions

From `Src/model_training.py`. Note `class_weight='balanced'`, which addresses the class imbalance discussed in Section 8.4.

```
models = {
    "Logistic Regression": LogisticRegression(
        max_iter=2000, class_weight="balanced", random_state=RANDOM_STATE),

    "Decision Tree": DecisionTreeClassifier(
        max_depth=8, min_samples_split=10, min_samples_leaf=5,
        class_weight="balanced", random_state=RANDOM_STATE),

    "Random Forest": RandomForestClassifier(
        n_estimators=300, max_depth=12, min_samples_split=5,
        min_samples_leaf=2, class_weight="balanced",
        random_state=RANDOM_STATE, n_jobs=-1),

    "Extra Trees": ExtraTreesClassifier(
        n_estimators=300, max_depth=12, min_samples_split=5,
        min_samples_leaf=2, class_weight="balanced",
        random_state=RANDOM_STATE, n_jobs=-1),

    "Gradient Boosting": GradientBoostingClassifier(
        n_estimators=200, learning_rate=0.05, max_depth=3,
        min_samples_split=5, min_samples_leaf=3,
        random_state=RANDOM_STATE),

    "HistGradient Boosting": HistGradientBoostingClassifier(
        max_iter=200, learning_rate=0.05, max_leaf_nodes=15,
        min_samples_leaf=10, l2_regularization=1.0,
        random_state=RANDOM_STATE),
```

```
}
```

B.3 Best model selection

From `Src/model_training.py`. The ranking order places F1 score before accuracy for the reasons given in Section 8.8.

```
successful = successful.sort_values(
    by=["F1 Score", "ROC-AUC", "Recall", "Accuracy"],
    ascending=False,
)

best_name = successful.iloc[0]["Model"]
best_model = trained_models[best_name]
```

B.4 Risk level assignment

From `Src/segmentation.py`. The composite risk score is banded into four business categories (Section 9.3).

```
def assign_risk_level(score: pd.Series) -> pd.Series:
    """Convert numerical risk score into business categories."""
    return pd.cut(
        score,
        bins=[-np.inf, 29.99, 59.99, 79.99, np.inf],
        labels=["Low", "Medium", "High", "Critical"],
    ).astype(str)
```

B.5 Strategic segment assignment

From `Src/segmentation.py`. Rules are applied in sequence so that later, more specific rules override earlier ones (Section 9.5).

```
segment = pd.Series("Standard Customer", index=df.index, dtype="object")

segment.loc[risk_score >= 80] = "Critical Retention"

segment.loc[(risk_score >= 60) & (risk_score < 80)
            & (value_percentile >= 70)] = "High Value At Risk"

segment.loc[(risk_score < 60) & (probability < 40)
            & (value_percentile >= 75)] = "Growth Opportunity"

segment.loc[(risk_score < 40)
            & (value_percentile >= 70)] = "Stable Valuable"

segment.loc[(risk_score < 30) & (probability < 30)] = "Loyal Customer"
```

B.6 Retention urgency and channel assignment

From `Src/recommendations.py`. Risk is combined with relative revenue exposure, and the engagement channel follows the resulting priority (Section 11.3).

```
result["Retention Urgency Score"] = (
    risk_score * 0.70 + value_factor * 0.30
).clip(0, 100).round(2)

result["Recommended Channel"] = np.select(
    [
        result["Retention Priority"].eq("Immediate"),
        result["Retention Priority"].eq("High"),
```

```
        result["Retention Priority"].eq("Monitor"),
    ],
    ["Personal Outreach", "Targeted Campaign", "Email / In-App"],
    default="Standard Engagement",
)
```

B.7 Defensive module import in the dashboard

From `app.py`. Each optional module sets an availability flag so that a failure disables one feature rather than the whole application (Section 5.6).

```
PREDICTION_AVAILABLE = False
PREDICTION_ERROR = None

try:
    from prediction_pipeline_backup import predict_customer_churn
    PREDICTION_AVAILABLE = True
except Exception as exc:
    PREDICTION_ERROR = exc
```

B.8 Orchestration of the analysis chain

From `app.py`. Prediction, segmentation and recommendation run in sequence, each guarded so that a later failure does not discard earlier results (Section 10.7).

```
with st.spinner("Running ChurnIQ machine-learning analysis..."):
    predictions = predict_customer_churn(data)

st.session_state.prediction_result = predictions

if SEGMENTATION_AVAILABLE:
    try:
        intelligence = create_customer_intelligence(predictions)
        st.session_state.intelligence_result = intelligence
    except Exception:
        st.session_state.intelligence_result = predictions

if RECOMMENDATIONS_AVAILABLE:
    try:
        recommendations = create_recommendations(intelligence)
        st.session_state.recommendation_result = recommendations
    except Exception:
        st.session_state.recommendation_result = intelligence
```

APPENDIX C — SAMPLE OUTPUT RECORDS

The following records were produced by executing the prediction, segmentation and recommendation modules on the reference dataset. They illustrate the output format of the complete pipeline. Customer identifiers are those present in the public IBM Telco dataset.

C.1 High-risk customers

Table C.1: *Sample of the highest-risk customers identified by the pipeline*

Customer ID	Ten.	Contract	Payment	Monthly	Prob.	Risk	Level	Segment	Priority
5419-JPRRN	1	M-to-M	Electronic check	101.45	96.6	96.1	Critical	Critical Retention	Immediate
9497-QCMMS	1	M-to-M	Electronic check	93.55	97.1	95.8	Critical	Critical Retention	Immediate
9300-AGZNL	1	M-to-M	Electronic check	94.00	97.0	95.8	Critical	Critical Retention	Immediate
0295-PPHDO	1	M-to-M	Electronic check	95.45	96.3	95.4	Critical	Critical Retention	Immediate
7024-OHCCK	2	M-to-M	Electronic check	93.85	96.5	95.3	Critical	Critical Retention	Immediate
4910-GMJOT	1	M-to-M	Electronic check	94.60	96.2	95.3	Critical	Critical Retention	Immediate
2720-WGKHP	2	M-to-M	Electronic check	94.00	96.3	95.2	Critical	Critical Retention	Immediate
7216-EWTRS	1	M-to-M	Electronic check	100.80	95.4	95.2	Critical	Critical Retention	Immediate

C.2 Low-risk customers

Table C.2: *Sample of the lowest-risk customers identified by the pipeline*

Customer ID	Ten.	Contract	Payment	Monthly	Prob.	Risk	Level	Segment	Priority
6928-ONTRW	72	Two year	Credit card (auto)	19.70	0.1	3.2	Low	Loyal Customer	Low
0464-WJTKO	72	Two year	Bank transfer (auto)	20.10	0.2	3.3	Low	Loyal Customer	Low
1052-QJIBV	71	Two year	Credit card (auto)	19.90	0.0	3.3	Low	Loyal Customer	Low
0404-AHASP	72	Two year	Credit card (auto)	19.70	0.3	3.4	Low	Loyal Customer	Low
3173-WSSUE	71	Two year	Credit card (auto)	19.70	0.1	3.4	Low	Loyal Customer	Low
7606-BPHHN	72	Two year	Credit card (auto)	19.80	0.4	3.4	Low	Loyal Customer	Low

C.3 Recommended actions

Table C.3: Distinct recommended actions generated by the pipeline

Risk Level	Recommended Action	Channel
High	Review payment experience and offer an easier payment option.	Targeted Campaign
Low	Maintain service quality and reward customer loyalty.	Standard Engagement
High	Launch an early-tenure onboarding and engagement campaign.	Targeted Campaign
Critical	Offer a personalized contract upgrade or retention incentive.	Personal Outreach
High	Provide personalized service outreach and retention incentives.	Targeted Campaign
Medium	Monitor behavior and provide proactive customer engagement.	Email / In-App
Medium	Introduce relevant upgrades and personalized offers.	Email / In-App
Low	Maintain engagement and monitor future churn signals.	Standard Engagement

Output columns produced by the complete pipeline

The final result set adds the following columns to the input dataset:

Table C.4: Columns added at each stage of the pipeline

Stage	Columns Added
Prediction (prediction_pipeline_backup.py)	Churn Prediction, Churn Probability, Risk Score, Risk Level, Prediction Method
Segmentation (segmentation.py)	Customer Risk Score, Risk Level (recomputed), Customer Value, Strategic Segment, Retention Priority, Priority Rank, Revenue At Risk
Recommendation (recommendations.py)	Recommended Action, Retention Recommendation, Retention Urgency Score, Recommended Channel

APPENDIX D — DASHBOARD SCREENSHOTS

Instructions for completing this appendix. The screen captures below must be taken from the running application and inserted in place of the corresponding placeholders. No screenshots have been generated artificially for this report. To capture them, start the application with `streamlit run app.py` from the repository root, open the URL shown in the terminal, navigate to each page listed below and capture the browser window. Load the default dataset and run the analysis from the Prediction Center first, so that the pages that depend on prediction results are populated.

D.1 Executive Overview

The landing page showing KPI tiles, the churn composition chart and generated insight cards.

[INSERT SCREENSHOT: Executive Overview]

Capture this page from the running application and insert the image here.

Figure D.1: *Executive Overview*

D.2 Dataset Center

The upload control, dataset preview, column information table and data-quality metrics.

[INSERT SCREENSHOT: Dataset Center]

Capture this page from the running application and insert the image here.

Figure D.2: *Dataset Center*

D.3 Dataset upload in progress

The file uploader after a CSV or XLSX file has been selected, showing the confirmation message.

[INSERT SCREENSHOT: Dataset upload in progress]

Capture this page from the running application and insert the image here.

Figure D.3: *Dataset upload in progress*

D.4 Churn Analysis

The predicted customer status pie chart and the churn probability histogram.

[INSERT SCREENSHOT: Churn Analysis]

Capture this page from the running application and insert the image here.

Figure D.4: *Churn Analysis*

D.5 Prediction Center

The prediction results table produced by the pipeline.

[INSERT SCREENSHOT: Prediction Center]

Capture this page from the running application and insert the image here.

Figure D.5: *Prediction Center*

D.6 Customer Segments

The strategic segment bar chart, composition pie chart and segment summary table.

[INSERT SCREENSHOT: Customer Segments]

Capture this page from the running application and insert the image here.

Figure D.6: *Customer Segments*

D.7 Risk Center

The risk distribution chart and the high-risk customer table with its download control.

[INSERT SCREENSHOT: Risk Center]

Capture this page from the running application and insert the image here.

Figure D.7: *Risk Center*

D.8 Model Performance

The best-model metric tiles, the six-model comparison table and the grouped comparison chart.

[INSERT SCREENSHOT: Model Performance]

Capture this page from the running application and insert the image here.

Figure D.8: *Model Performance*

D.9 Customer Explorer

The customer search interface and the detail view for an individual customer.

[INSERT SCREENSHOT: Customer Explorer]

Capture this page from the running application and insert the image here.

Figure D.9: *Customer Explorer*

D.10 Recommendations

The business summary metrics and the per-customer recommended actions.

[INSERT SCREENSHOT: Recommendations]

Capture this page from the running application and insert the image here.

Figure D.10: *Recommendations*

D.11 Reports

The executive KPI summary, business exposure section and the CSV download controls.

[INSERT SCREENSHOT: Reports]

Capture this page from the running application and insert the image here.

Figure D.11: *Reports*

D.12 About

The methodology section describing the ten-stage workflow.

[INSERT SCREENSHOT: About]

Capture this page from the running application and insert the image here.

Figure D.12: *About*